

What does an end-of-life alarm score measure? Separating prediction accuracy, alarm timing, maintenance cost and transfer

ARTICLE INFO

Keywords:

Remaining useful life
Predictive maintenance
Alarm evaluation
Alarm timing
Decision-oriented metric
Distribution shift
Lithium-ion battery

ABSTRACT

End-of-life alarms trigger replacements but are often judged by a single acceptance score. We ask what such a score measures and whether optimising it yields good maintenance decisions. On eight run-to-failure datasets, with a deliberately simple alarm and training-only selection, we separate long-range prediction, near-failure estimation, alarm timing, maintenance cost and transfer. With matched training and calibration, one fixed-time model placed 24.8% of PHM08 estimates in an asymmetric window at half of life and 85.3% at 90% of life; the self-timed alarm reached 91.7%, no more than its regressor at 90% of life, and FD004 repeated this. A constant estimate at the alarm times scored within 3.6 points of the model's estimate, so the score mainly rewarded timing and concealed notice and cost. Window-tuned alarms beat age replacement in most cost settings on six of seven datasets, but no tuning rule was cheapest everywhere: under the re-implemented metric of Kamariotis et al., their threshold policy on our regressor won on FD001 and lost on FD002 and FD003. With a battery batch held out, accuracy fell from 95.1% to 79.7% (63.4–84.6% across time scales) and a capacity trend did better; on bearings the alarm failed. These properties should be reported separately.

1. Introduction

A prognostic model creates value only through the maintenance decisions it triggers. For a component that is replaced once, near the end of its life, that decision is usually an end-of-life alarm: a policy watches the unit cycle by cycle, stays silent while the unit looks healthy and raises one alarm, which schedules a replacement. Whether a prognostic method is worth using therefore depends on when its alarms fire, how much notice they give, what the resulting replacements and failures cost, and whether this holds for units other than those it was developed on. These are reliability questions in the sense of classical replacement theory, in which a condition-based policy must at least beat age replacement [1, 2].

Data-driven prognostics estimates the remaining useful life (RUL) of a component from its condition-monitoring history [3, 4]. On the standard turbopfan benchmarks the estimates are usually scored at times chosen by the evaluator: the last cycle of truncated test trajectories, every cycle, or fixed fractions of life [5]. At least four quantities are then easily conflated. **Long-range prediction** is the accuracy of an estimate made far from failure. **Near-failure estimation** is its accuracy when little life is left. **Alarm timing** is the distribution of the remaining life at the moment a policy acts, which is the notice that maintenance actually receives. **Maintenance-policy value** is the long-run cost of the replacements and failures that the alarms produce. A fifth property, **transfer**, asks whether any of these survives a change of operating conditions, test campaign or asset type. The quantities are related, but good performance on one need not imply good performance on another.

End-of-life alarms are often judged by a single acceptance score that mixes these quantities. Prognostic

scores and losses commonly penalise late predictions more than early ones [6, 7]. This paper studies one such score: an asymmetric acceptance window for a single alarm, under which an alarm is correct if the true remaining life lies between 2 cycles below and 13 cycles above the reported estimate, and a unit that fails without an alarm counts as late. The score depends on the estimate, on when the alarm fires and, implicitly, on the notice given; it says nothing directly about cost. A high window score could therefore reflect accurate remaining-life estimation, well-timed triggering, or both, and it is not obvious which.

Alarm-based maintenance and decision-oriented evaluation are established. de Pater et al. [8] trigger fleet maintenance scheduling from alarms on probabilistic RUL prognostics, and Kamariotis et al. [9] evaluate prognostic algorithms, and tune the policies built on them, by the increase in long-run maintenance cost that they induce relative to perfect prognostics. Neither study is designed to answer four questions that matter for interpreting an alarm score: how much of the apparent advantage of a self-timed alarm over fixed-time prediction comes from the alarm itself rather than from the time at which accuracy is measured; whether the window score measures the remaining-life estimate or the timing of the alarm; whether alarms tuned to the window are also good maintenance policies, including under a decision-oriented metric; and whether performance validated within familiar conditions holds on a held-out test batch or another asset type.

We address these questions with a controlled evaluation on eight run-to-failure datasets: the PHM08 engines, C-MAPSS FD001–FD004, N-CMAPSS, lithium-ion cells cycled to failure in three batches, and rolling bearings. A deliberately simple and transparent alarm,

a gradient-boosted regressor with a threshold trigger, serves as the measuring instrument, so that the analysis concerns the evaluation rather than a model. Every setting is selected on training units only, and the design changes one element at a time: matched fixed-time and self-timed variants with the same training data and calibration; controls that keep the alarm times but replace the estimate, or keep the trigger but replace the regressor; a renewal-reward cost model with age replacement and directly cost-tuned policies; a re-implementation of the metric M of Kamariotis et al. [9] with their decision grid and their policies on our regressor; and held-out batches and other asset types. The study answers four questions:

- (Q1) How do long-range prediction, near-failure estimation and alarm timing differ, and how much of the apparent advantage of a self-timed alarm over fixed-time prediction remains when training data and calibration are matched?
- (Q2) What does the asymmetric alarm window reward, and what does its score conceal?
- (Q3) Does optimising window accuracy produce good maintenance policies, compared with age replacement, with alarms tuned directly to cost and, under the metric M , with the probability-threshold policies of Kamariotis et al. [9]?
- (Q4) What happens to accuracy and cost when evaluation moves from familiar conditions to a held-out test batch or to a different asset type?

The answers are four findings:

- (i) **Observation time, not self-timing, explains most of the apparent advantage of alarms over fixed-time prediction.** Under matched training data and cross-fitted calibration, the same fixed-time model placed 24.8% of its estimates in the window at half of life and 85.3% at 90% of life on PHM08 (FD004: 18.9% and 66.7%), whereas the self-timed alarm was no more accurate than its own regressor read at 90% of life (91.7% against 91.7%; FD004: 82.7% against 82.3%). Self-timing makes near-failure accuracy available without knowing the life; it did not make the estimates more accurate.
- (ii) **The window score is driven mainly by alarm timing.** Replacing the model's estimate at the alarm by one constant per fold changed the score by at most 3.6 percentage points on each of seven datasets, and the shift that tuning selects acts mainly by moving the alarm. The quality of the timing is not trivial: alarms timed by a linear health indicator, even when the indicator was calibrated to cycles and given the same trigger,

scored significantly lower on PHM08 and FD001–FD004. The score does not reveal how much notice the alarms give.

- (iii) **Optimising the window is not equivalent to optimising maintenance cost.** Window-tuned alarms were cheaper than optimal age replacement in most cost settings on six of seven datasets, but not on N-CMAPSS, and alarms tuned directly to cost were cheaper on some datasets and more expensive on others. Under the metric M , which we relate exactly to our cost ratio, the probability-threshold policy of Kamariotis et al. [9] built on our regressor had a lower M than our window-tuned alarm on FD001 (5.69% against 8.08% on FD001) and a higher M on FD002 and FD003; no policy family was best on every dataset.
- (iv) **Performance validated within familiar conditions did not transfer.** On the battery cells, with fully nested preprocessing, the recipe's in-window rate fell from 95.1% under cross-validation by cell to 79.7% with a whole batch held out (63.4–84.6% across fixed time scales, 74.8% with the time scale chosen by inner cross-validation, and 69.9% and 65.0% under two earlier conventions), while a capacity-trend baseline was more accurate and cheaper in most cost settings. The recipe reached 68.7% on N-CMAPSS and 0 of 17 on rolling bearings.

Together, the findings show that long-range accuracy, near-failure accuracy, alarm timing, maintenance-policy value and transfer are distinct properties of an end-of-life alarm, and that a single window score does not stand in for the others. The paper proposes no new alarm rule or metric. Its contribution is the evaluation design and the evidence it produces, from which we derive a recommendation for what alarm studies should report (Section 9.5). Table 1 collects the key numbers by property, with the analysis and status behind each.

Section 2 reviews related work; Section 3 defines the task, the five properties and the cost model; Section 4 describes the alarm used as the instrument; Section 5 gives the experimental design and the status of each analysis; Sections 6–8 report the results for Q1–Q2, Q3 and Q4; Section 9 discusses them; and Section 10 concludes.

2. Related work

2.1. Remaining-life prediction and its evaluation

The C-MAPSS simulations and the PHM08 challenge data [6] are the standard engine benchmarks. The three winning PHM08 approaches were a similarity-based method [10], a recurrent network [11] and an ensemble of neural networks

Table 1

Key numbers by property of the alarm, with the analysis behind each. A: long-range prediction; B: near-failure estimation; C: alarm timing; D: maintenance-policy value; E: transfer. Status: see Table 3; every exploratory analysis was added after the main results were known. Battery conventions (Section 5.2): A, $\kappa = 3$ from all cells, as in the protocol; B, κ from the outer training cells, task in blocks; C, fully nested, task in cycles (primary).

Property	Result	Value	Analysis and status	Where
A vs B	Same calibrated fixed-time model at half and at 90% of life (PHM08)	24.8% vs 85.3% in the window	Matched, cross-fitted; exploratory	Table 5
B vs C	Self-timed recipe vs its regressor at 90% of life (PHM08)	91.7% vs 91.7% ($p = 1.000$)	As above	Table 5
A–C	The same two comparisons on FD004	18.9% vs 66.7%; 82.7% vs 82.3% ($p = 1.000$)	As above; plan hashed before the run	Table 5
B vs C	Regressor at the oracle time of 20 cycles left vs self-timed alarm (FD004)	89.2% vs 82.7% ($p = 0.029$)	As above	Table 5
C	Constant estimate at the recipe's alarm times: largest difference from the model's estimate	3.6 points (7 datasets, none significant)	Timing control; exploratory (battery: convention B)	Table 7
C	Health-indicator alarm calibrated to cycles, recipe's trigger: PHM08 and FD001–FD004 / N-CMAPSS	48.0–55.8% (recipe 82.7–96.0%) / 69.7% (recipe 68.7%)	Exploratory; plan hashed before the run	Table 7
D	Window-tuned alarms significantly cheaper than age replacement (cells of 16)	6 of 7 datasets in ≥ 12 cells; N-CMAPSS in 0	Renewal–reward model; exploratory (battery: convention C)	Table 8
D	M on FD001 at $c_p/c_c = 0.1$ ($\Delta T = 10$): their policy 1 on our regressor / our 1-SE / our window-tuned alarm	5.69% / 7.04% / 8.08%	Re-implementation of M , five-fold; plan hashed before the run	Table 9
D	Lowest M at $c_p/c_c = 0.1$, by dataset (FD001–FD004)	their policy 1 (P-res) on FD001 (5.69%); our window-tuned alarm on FD002 (4.85%); their policy 1 (P-cl) on FD003 (3.46%); our window-tuned alarm on FD004 (8.44%)	As above	Table 9
D	Their policy 1 (P-res) beats our window-tuned alarm at 0.1 (paired bootstrap)	FD001; unclear on FD004; ours wins on FD002, FD003	As above	Table 10
D	Fold–ratio cells with ≥ 1 test failure at $c_p/c_c \leq 0.1$, FD001–FD004 (of 80): policy 1 P-cl optimised vs $p = c_p/c_c$; STW plain vs 1-SE	12 vs 19; 24 vs 4	Fragility analysis; plan hashed before the run	Section 7.5
D	Window-tuned alarm vs policy 1 (P-res, optimised), cells with ≥ 1 test failure at $c_p/c_c \leq 0.1$: five-fold FD001–FD004 (of 80) / thirty FD001 splits (of 120)	0 vs 4 / 12 vs 4	As above	Section 7.5
E	Battery recipe: cross-validation by cell / one batch held out (convention C)	95.1% / 79.7% (held out: fixed κ 63.4–84.6%, κ by inner CV 74.8%, convention A 69.9%, B 65.0%)	Corrected (post hoc)	Table 11
E	Capacity-trend baseline: cross-validation / batch held out (convention C)	100.0% / 98.4%	Exploratory	Table 11
D, E	Trend cheaper than the window-tuned recipe with a batch held out (cells of 16; convention C)	window-tuned trend 12, cost-tuned trend (1-SE) 12	Fully nested cost; plan hashed before the run	Table 12
E	N-CMAPSS / rolling bearings, $H = 20$	68.7% / 0 of 17	Pre-specified (bearings: time scale corrected)	Table 13

filtered by a Kalman filter [12]. Later work moved to long short-term memory [13] and convolutional networks [14], usually with a remaining-life target capped near 125 cycles. N-CMAPSS adds realistic flight profiles [15]. On tabular inputs, tree ensembles remain strong [16, 17]. Severson et al. [19] cycled commercial lithium-ion cells to failure, and the PRONOSTIA platform produced run-to-failure vibration records of rolling bearings [18]. On the evaluation side, Ramasso and Saxena [5] showed how strongly reported scores depend on the choice of cut-off points and metrics. Saxena et al. [20] introduced metrics that follow a prediction over time, such as the prognostic horizon and α - λ accuracy, and asymmetric losses that penalise late predictions are common [7]. These metrics evaluate the estimate at times fixed by the evaluator; they do not separate the estimate from the timing of a decision that the policy itself chooses.

2.2. From prognostics to maintenance decisions

Age replacement, which uses no condition information, is the classical benchmark for any condition-based policy [1, 2]. Degradation models with Bayesian updating yield residual-life distributions [21], and dynamic predictive-maintenance frameworks act on predicted failure probabilities [22, 23] or learn policies from probabilistic prognostics by reinforcement learning [24]. de Pater et al. [8] propose a fleet-level scheduling framework for aircraft in which periodically updated RUL prognostics of a convolutional network trigger alarms, maintenance is scheduled only after an alarm, a safety factor guards against prognostic error, and an integer linear program assigns aircraft to maintenance slots. Kamariotis et al. [9] propose a metric that assesses data-driven prognostic algorithms by their effect on downstream predictive-maintenance decisions: the relative increase in long-run maintenance cost per unit time, under a given decision setting and policy, compared with perfect prognostics. Decisions are taken at discrete times, and their policies act on the predicted probability of failure or on the whole predictive distribution of the remaining life. They use the metric to compare algorithms and to tune policies and hyperparameters on C-MAPSS FD001 (Section 7.2). Xie et al. [25], in a preprint, train the prognostic model itself against downstream maintenance cost on simulated turbofan (C-MAPSS) data.

2.3. Position of this study

These studies evaluate prognostics through decisions; the present study asks what an alarm score measures and whether it can stand in for such an evaluation. de Pater et al. [8] design an alarm-triggered scheduling system and evaluate its fleet cost; we take the alarm rule as given and dissect a common kind of acceptance criterion for single alarms. Kamariotis

et al. [9] recommend scoring prognostics by decision cost; we re-implement their metric, test window-tuned alarms under their conventions against their policies built on the same regressor, and use the same kind of cost evaluation to test whether tuning to an operational window is an adequate substitute for tuning to cost. Neither study separates observation time, calibration, target design and alarm timing under matched training data, or examines what happens to window accuracy and cost when a whole test batch is held out. Because such comparisons are easily biased by model selection on evaluation data [26] and by leakage [27], every setting here is selected on training units only. Three datasets were analysed under protocols fixed in advance [28], and every other analysis is labelled by how it arose (Section 5.3).

3. Problem formulation

3.1. First-warning task

A unit i is observed at cycles $t = 1, \dots, L_i$, where L_i is its life, unknown in service. The true remaining life at cycle t is $r_t = L_i - t$. A warning policy maps the history up to t to either *silent* or *warn*, together with an estimate $\hat{y}_t$. The first-warning time is

$$t^* = \min\{t < L_i : \text{the policy warns at } t\}, \quad (1)$$

and the error of the warning is $e = \hat{y}_{t^*} - r_{t^*}$. A warning is **correct** if $-13 \leq e \leq +2$, that is, exactly when

$$\hat{y}_{t^*} - 2 \leq r_{t^*} \leq \hat{y}_{t^*} + 13. \quad (2)$$

It is **too early** if $e < -13$ and **too late** if $e > +2$; a unit that fails without a warning is **missed** and counted with the late outcomes. The window score is the share of units whose first warning is correct (the in-window rate). We also report the late-or-missed and too-early rates, the true remaining life at the alarm (the notice actually given), and the share of alarms raised with $r_{t^*} \geq 13$. Only those alarms can be too early, unless the estimate is negative, so a policy that warned only in the last few cycles would meet the window trivially. The window was fixed by an operator's requirement before this study; its early limit coincides with the early constant of the PHM08 score [6], and its late limit is much stricter than that score's late constant of 10.

The warning level H expresses how much notice is wanted: the policy warns when it believes that about H cycles are left. $H = 20$ is the primary level, designated on PHM08 as the smallest of $\{10, 15, 20, 30, 40\}$ at which more than 90% of alarms were raised with at least 13 cycles left (Section 5.3).

3.2. Five properties of an end-of-life alarm

We evaluate five properties separately. (A) **Long-range prediction**: the accuracy of $\hat{y}_t$ at times chosen independently of the policy far from failure, for example half of life. (B) **Near-failure estimation**: the accuracy of $\hat{y}_t$ when r_t is small, for example at 90% of life or at the cycle where $r_t = H$. (C) **Alarm timing**: the distribution of r_{t^*} produced by the policy, that is, the notice given. (D) **Maintenance-policy value**: the long-run cost rate of the replacements and failures that the alarms produce (Section 3.3). (E) **Transfer**: whether (A)–(D) hold on a test population that differs from the training population, such as a held-out test batch or another asset type. The window score of Section 3.1 is computed at the alarm, so it depends on (B) and (C) jointly and on (E) through the evaluation design; it does not measure (D).

A fixed fraction of life is not known in service, but evaluating a model retrospectively at such a time is not leakage. It would be leakage if the fraction, the life or a rule derived from them were given to the model as an input or used to train or select it; the fixed-time models of Section 6.1 received none of these. Conversely, the first-warning task is not easier merely because the policy chooses its time: the choice must be made from the history alone, and a late or missed alarm is a failure.

3.3. Maintenance cost

Alarms are priced in a renewal–reward model [1]. An alarm at cycle t^* schedules a planned replacement ℓ cycles later. If $r_{t^*} > \ell$ the replacement happens at $t^* + \ell$ and costs 1; otherwise, or if no alarm is raised, the unit fails at L and the failure costs ρ . An alarm with exactly ℓ cycles left counts as a failure. Units are replaced by new ones, so the long-run cost per cycle is estimated by the total cost divided by the total cycles of use over the test units. We use lead times $\ell \in \{0, 5, 10, 15\}$ cycles for the engines and $\{0, 15, 30, 45\}$ charge–discharge cycles for the batteries, and cost ratios $\rho \in \{5, 10, 20, 50\}$. The ratios are illustrative, not measured. Optimal age replacement replaces every unit at the age T that minimises the cost rate on the training lives; it needs no lead time because the replacement age is known in advance. Perfect foresight (replacement at $L - 1$, cost 1) and run-to-failure bound the range, and every cost rate is reported relative to perfect foresight. With $\ell = 0$ this ratio is closely related to the metric M of Kamariotis et al. [9], but it equals $1 + M$ only when decisions can be taken at every cycle; Section 7.2 gives the exact relation for a coarser decision grid.

3.4. Physical units

Battery and bearing lives are several times longer in their native units than engine lives, so the models work in blocks of κ consecutive charge–discharge cycles or vibration snapshots (Section 5.2). The task itself is defined in physical units. For the batteries it is $H = 60$

cycles and the window $[-39, +6]$ cycles, the engine task scaled by three; for the bearings it is $H = 140$ snapshots and $[-91, +14]$ snapshots. A model with block length κ uses the warning level, window, lead times, caps and shifts divided by κ , so that every acceptance decision, alarm cost and setting has the same meaning whatever κ is; the remaining life at the alarm and the cycles of use are reported in cycles. We write “unit” for an engine, cell or bearing.

4. The alarm used as the instrument

Fig. 1 shows the self-timed warning (STW) recipe. It is built from standard parts so that its behaviour can be dissected; its online part runs at every cycle of a unit in service, and its offline part fixes four settings $\theta = (c, m, k, s)$ on training units only.

Normalisation and features. Each sensor is z-scored within its operating regime with statistics of the training units, and sensors that are constant within any regime are dropped (15 sensors remain on PHM08, FD001 and FD002, 16 on FD003 and FD004; N-CMAPSS uses the six temperature and shaft-speed channels fixed in its protocol). At cycle t the encoder uses cycles $1, \dots, t$ only: an early-life baseline (the mean of the first $\min(20, t)$ cycles), exponentially weighted means with spans 10 and 30 of the baseline-removed signals, least-squares slopes over the last 15 and 40 cycles, and the age t (Supplementary Section S4 checks causality).

Regressor. A histogram gradient-boosted regressor [29] with absolute-error loss (500 iterations, learning rate 0.05, 31 leaves, at least 40 samples per leaf, seed 0) is trained on all cycles of the training units with the target

$$y_t = \min(r_t, c), \quad c \in \{40, 60, 90\}. \quad (3)$$

Capping the target is standard [11]; a low cap concentrates the loss on the last c cycles, where the alarm is raised.

Trigger. The estimate is smoothed causally and shifted, $\tilde{y}_t = \text{EWM}_m(\hat{y})_t + s$, with span $m \in \{1, 3, 6\}$ ($m = 1$: no smoothing) and $s \in \{-12, -11.5, \dots, +8\}$, and the alarm is raised at

$$t^* = \min\{t : \tilde{y}_\tau \leq H \text{ for all } \tau = t - k + 1, \dots, t\}, \quad (4)$$

with debounce $k \in \{1, 2, 3, 5\}$; the reported estimate is $\tilde{y}_{t^*}$.

Nested selection. The training units of each outer fold are split into five inner folds. Regressors fitted on four inner folds, which also supply the normalisation statistics, predict every cycle of the fifth, giving out-of-fold predictions for all training units. For each H , the grid (caps with $c \geq H + 20$, three spans, four debounce

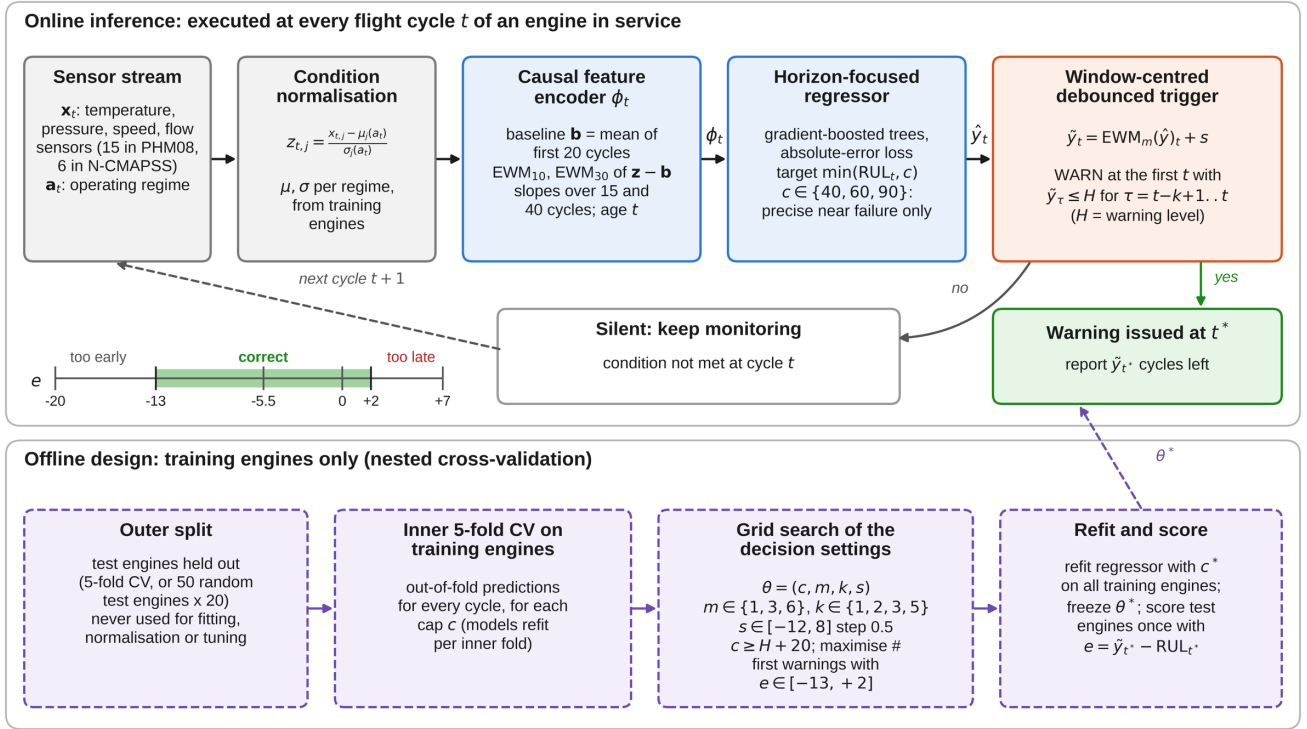


Fig. 1: The self-timed warning (STW) recipe. Online (top): regime-normalised sensors are encoded causally and mapped to an estimate $\hat{y}_t$, which is smoothed and shifted to $\tilde{y}_t$; an alarm is raised the first time $\tilde{y}$ stays at or below H for k cycles. Offline (bottom): $\theta = (c, m, k, s)$ is chosen on inner out-of-fold predictions of the training units and frozen; test units are scored once. Inset: the scoring window for $e = \tilde{y}_{t^*} - r_{t^*}$.

values and 41 shifts) is searched for the setting with the most correct out-of-fold first warnings; ties go to the smaller $|s|$. The regressor is then refitted on all training units, θ is frozen, and each test unit is processed once. Supplementary Section S13 lists every setting.

4.1. The role of the shift

Let $\varepsilon = \text{EWM}_m(\hat{y})_{t^*} - r_{t^*}$ be the unshifted error at the alarm, so that $e = \varepsilon + s$, and write the window as $[a, b] = [-13, +2]$, of width $w = 15$. The following elementary result explains why tuning to the window favours a negative shift. It is stated for a **fixed** error distribution.

Proposition 1. Let ε have a continuous distribution F that does not depend on s (assumption A1). Then the probability that $e = \varepsilon + s$ lies in $[a, b]$ is

$$P(s) = F(b - s) - F(a - s). \quad (5)$$

(i) $P(s) \leq C_w(F) = \sup_x [F(x + w) - F(x)]$ for every s . (ii) If F has a density that is unimodal and symmetric about its median m , $P(s)$ is maximised at

$$s^* = (a + b)/2 - m = -5.5 - m, \quad (6)$$

with maximum $\Pr(|\varepsilon - m| \leq 7.5)$. The proof is in Supplementary Section S2.

The result implies that an unbiased estimator should be shifted by about -5.5 cycles, because the window is centred at -5.5 . It is **not** a guarantee for the sequential policy: changing s changes the stopping time t^* and therefore the distribution of ε , so in practice F depends on s . Section 6.2 measures how far the alarm moves when the shift changes, which is the mechanism by which tuning to the window acts on alarm timing.

5. Experimental design

5.1. Data

Table 2 lists the data. We use the 218 run-to-failure engines of PHM08, the training sets of C-MAPSS FD001–FD004 [6], and N-CMAPSS reduced to one row per engine and flight [15]. The battery data are commercial LFP/graphite cells (A123 APR18650M1A, 1.1 Ah) cycled to failure under 66 fast-charging protocols in three batches that also differ in their rest periods [19]. The model sees seven per-cycle quantities (discharge and charge capacity, internal resistance, average, minimum and maximum temperature, charge time), z-scored as one regime as fixed in the protocol. The bearing data are the 17 complete records of the PRONOSTIA platform under three load and speed conditions [18], reduced to root mean square, peak, kurtosis and crest factor of the horizontal and vertical acceleration of each snapshot. The engine datasets

Table 2

Datasets. Life: cycles (battery: charge–discharge cycles; bearings: vibration snapshots) to failure of the run-to-failure units. Time unit: the unit in which the models work; κ is computed from training units only (Section 5.2).

Dataset	Units	Rows	Life, min–max (median)	Regimes	Time unit of the model	Status (Table 3)
PHM08	218	45,918	128–357 (209.5)	6	cycle	development
FD001	100	20,631	128–362 (199)	1	cycle	additional evaluation
FD002	260	53,759	128–378 (199)	6	cycle	additional evaluation
FD003	100	24,720	145–525 (220.5)	1	cycle	additional evaluation
FD004	249	61,249	128–543 (234)	6	cycle	additional evaluation
N-CMAPSS	99	7,473	48–100 (75)	3 (flight class)	flight cycle	pre-specified
Battery [19]	123 cells	95,276	148–1935 (731)	1	block of κ cycles ($\kappa =$ 2–6)	pre-specified; time scale corrected
Bearings [18]	17 bearings	24,889	230–2803 (1428)	3 (load/speed)	block of κ snapshots ($\kappa \in \{4, 6, 7, 9\}$)	pre-specified; time scale corrected

C-MAPSS and PHM08: training sets only (complete trajectories); the truncated official test sets are not used. N-CMAPSS: one row per engine and flight (mean of the 1 Hz signals), 99 engines of nine subsets. Battery: 123 of 140 cell records after exclusions fixed before any model was run (split tests, censored cells and two records without an official life; Supplementary Section S5); life = first cycle below 80% of the nominal 1.1 Ah. Bearings: one row per 0.1-s snapshot recorded every 10 s; eight features per snapshot. PHM08, FD001–FD004 and N-CMAPSS come from one simulator family (C-MAPSS).

vary the fault modes and operating regimes within one simulator family; the batteries add a held-out test batch as a change of test campaign; the bearings add a different asset type.

5.2. Time scale of the battery and bearing data

The protocols set the block length to $\kappa = \max(1, \text{round}(\text{median life} / 209.5))$, 209.5 cycles being the median PHM08 life, so that the grids designed for engine lives could be reused unchanged. As executed under the protocols, the median was taken over all units, including later test units, so held-out lives could influence the time scale. We report those results for provenance only and base every interpretation on time scales computed from training units. Three conventions appear: **A**, $\kappa = 3$ from all cells, as executed under the protocol (provenance only); **B**, κ from the outer training cells, inherited by the inner folds, with the task in blocks; and **C**, fully nested preprocessing with the task in cycles, in which every inner training subset builds its own blocks, z-score statistics and regressor with κ computed from that subset only, and no held-out life enters any time scale or setting. Convention C is primary for every battery result, including the cost analysis. Two battery analyses keep convention B and are labelled so: the timing controls and the shift study (Section 6.2). The held-out-batch estimate is also reported with κ chosen by inner cross-validation and with κ fixed at 2–6. The bearing analyses use κ from the outer training set, and a sensitivity analysis holds the physical task fixed for $\kappa \in \{3, 5, 7, 10, 14\}$. Supplementary Section S19 gives the details of the nesting and of how an operator would choose κ , and Section S16 the history of the correction.

5.3. Status of each analysis

Several analyses in this study were added in response to earlier results, including results that weakened the first conclusions. Rather than present all analyses as equivalent, we label each by how it arose (Table 3). The labels let a reader weigh each result by how independent it is of the choices that produced it: the development dataset is likely to be optimistic, the pre-specified analyses test hypotheses fixed before the data were analysed, and the exploratory analyses describe the method rather than confirm it.

The model and grids were developed on PHM08 after fixed-time results of several models were known, and the C-MAPSS datasets were then run once with unchanged code. $H = 20$ was designated afterwards by the criterion of Section 3.1 computed on PHM08 (95.4% of alarms raised with at least 13 cycles left at $H = 20$, against 65.6% at $H = 15$). Three protocols were then written in sequence, each after the previous result was known. Each fixed the data, the time-scale rule (for aggregated data), the unchanged recipe, two competitors under the same tuning (an age-only regressor and a multilayer perceptron with a Kalman filter), five-fold cross-validation by unit (seed 2026), and four hypotheses at $H = 20$: in-window rate $\geq 85\%$; late-or-missed rate $\leq 10\%$; higher in-window rate than age-only and than the MLP with Kalman filter (exact McNemar test, two-sided $p < 0.05$). The bearing protocol added a fifth: that a theory-guided shift rule (STW-T, which sets the median out-of-fold error to the window centre) would be no more than 5 points worse than the standard rule. A hash shows that a protocol has not changed since it was hashed, not when it was hashed; the dates rest on the authors' records and file time stamps. N-CMAPSS had been used earlier in the project for fixed-time prediction with other models, although no warning rule had been fitted to it.

Table 3

Status of every analysis. Pre-specified: protocol written and hashed (SHA-256) before the model or a competitor was run on the data; the hashes are held by the authors and were not deposited with a registry.

Status	Analyses	What the status means
Development	PHM08 (model, grids, warning levels and primary level $H = 20$)	Choices made while looking at PHM08 results; estimates may be optimistic
Additional evaluation, not pre-specified	C-MAPSS FD001–FD004 (run once with unchanged code)	Code fixed before the run, but $H = 20$ was designated after these results were known
Pre-specified	N-CMAPSS (protocol 2026-09-30), battery (2026-09-30), bearings (2026-10-01): data, time scale rule, model, grids, competitors, splits, levels and hypotheses	Protocol hashed before the analysis; N-CMAPSS had been used earlier for fixed-time prediction
Corrected (post hoc)	Battery and bearing analyses with training-only time scales; fully nested battery analysis with a physical task; κ sensitivity controls; health-indicator alarm; battery capacity baselines; identical-protocol competitors and networks with matched inputs; ablation; oracle-timed control; shift study; engine cost analyses; other windows; 14-channel N-CMAPSS; theory-guided shift on seen data; thirty random 50-engine test sets; bearing onset diagnosis	Deviation from the hashed protocols, made after their results were known
Exploratory (post hoc)	Matched fixed-time decomposition on PHM08; timing controls; health-indicator alarm; battery capacity baselines; identical-protocol competitors and networks with matched inputs; ablation; oracle-timed control; shift study; engine cost analyses; other windows; 14-channel N-CMAPSS; theory-guided shift on seen data; thirty random 50-engine test sets; bearing onset diagnosis	Describe the method; not confirmations
Exploratory (post hoc), plan hashed before the run	Comparison with the metric M of Kamariotis et al. on FD001 (plan of 2026-10-06, amended before the run) and FD002–FD004, with the fragility analysis of threshold optimisation; calibrated health-indicator alarm; fully nested battery cost analysis; matched decomposition on FD004 (second plan of 2026-10-06)	Analysis fixed before it was run, after the main results were known; not confirmations

5.4. Comparisons

Fixed-time versus self-timed (Q1). The matched comparison uses one cross-fitting procedure for every variant. Inner five-fold out-of-fold predictions of the outer training engines (seed 100 + fold, normalisation from each inner training subset) are used for every calibration or selection step, and the final model is refitted on all outer training engines. Outer test engines never enter fitting, calibration or selection. The variants change one element at a time. The **observation time** is 50% or 90% of life, the cycle with exactly 20 true cycles left (an oracle time that cannot be deployed), or the policy's own alarm. The **regressor** is either a fixed-time gradient-boosted model (no target cap, features of seven temperature and speed sensors) or the recipe's capped regressor. **Calibration** is a shift chosen on the out-of-fold errors at the same observation time; the trigger variants choose span, debounce and shift on out-of-fold alarms. The cross-fitted variants were run on PHM08 and repeated on FD004, which has two fault modes and six regimes, with the same seeds. For reference, eleven fixed-time models were also evaluated under a fit-75%/calibrate-25% protocol at 30, 50, 70 and 90% of each test engine's life (Supplementary Section S1), and a matched 75%/25% comparison of the fixed-time model and the recipe is reported.

Timing controls (Q2). A *constant-estimate control* keeps the recipe's alarm times and replaces the reported estimate by one constant per fold, chosen on training out-of-fold alarms; it shows whether the estimate adds information beyond the alarm time. A *health-indicator threshold alarm* has no remaining-life regressor: a linear health indicator, in the spirit of

Wang et al. [10], is fitted to separate the first 30 from the last five cycles of each training unit, and the alarm is raised when its smoothed value stays above a threshold for k cycles, reporting a constant in $[H - 6, H]$. A *calibrated health-indicator alarm* tests whether a fairer indicator-based opponent closes the gap: the same linear indicator is mapped to cycles to failure by an isotonic, decreasing regression of $\min(r, c)$ on its inner out-of-fold values ($c \in \{40, 60, 90\}$), and the mapped series goes through the recipe's trigger with the recipe's grids and selection rule. A *shift study* varies the shift on test units around its selected value to see what tuning acts on.

Maintenance policies (Q3). In the renewal-reward model of Section 3.3, the window-tuned alarms are compared with optimal age replacement, with the health-indicator alarm tuned to cost, and with policies that tune the same regressor directly to the training cost rate, with and without a one-standard-error safeguard against selection noise, and with a cost-tuned 10% quantile alarm. Under the metric M they are compared with policies 1 and 3 of Kamariotis et al. [9] built on the same regressor (Section 7.3).

Transfer (Q4) and baselines. The battery cells are evaluated under cross-validation by cell, in which every batch is represented among the training cells, and with each of the three batches held out in turn. Two baselines use the measured discharge capacity directly: a threshold alarm on smoothed block-mean capacity that reports a constant, and a capacity trend that extrapolates a least-squares line through the capacity of the last 30, 60 or 120 cycles to the end-of-life capacity of 0.88 Ah and feeds the extrapolated remaining life to the same trigger. Neither fits

a model, so their settings are chosen on the training cells themselves; they are evaluated on the window, on the notice they give and on cost. On N-CMAPSS and the bearings the protocols fixed an age-only regressor and a multilayer perceptron with a Kalman filter as competitors. Re-implementations of the three winning PHM08 approaches and three neural networks given the recipe's inputs, together with a component ablation, are reported in Supplementary Section S15; they show that the recipe is a competitive instrument on PHM08, not that it is the best model.

5.5. Policies and their information sets

Table 4 lists every policy compared, the information it uses and how its settings are selected. All selection uses training units only: out-of-fold predictions where a regressor is involved, the training series themselves where nothing is fitted. Outer test units are scored once. The window-tuned policy uses cost only to choose its warning level among five values; every other setting is chosen by the window count. The cost-tuned policies choose all their settings by the training cost rate. The two grids are of similar size (about 1,500 settings per warning level against 2,160), so the comparison concerns the selection criterion rather than the size of the search.

5.6. Statistics and what the uncertainty covers

Proportions carry Clopper–Pearson 95% intervals [30]. Paired comparisons of in-window rates use the exact McNemar test [31]. Holm's correction [32] is applied over the seven competitors at each level of the identical-protocol comparison, over the nine ablation variants, and, for the health-indicator alarm, over the seven datasets of the timing-control table (six engine datasets for its calibrated version). The constant-estimate comparisons are reported unadjusted, since none was significant even before adjustment. The pre-specified hypothesis tests are reported unadjusted, as fixed in the protocols. A non-significant difference is not read as equivalence, because no equivalence margin was specified. Ratios of cost rates carry paired 95% intervals from 2,000 bootstrap resamples of test units with the fitted policies held fixed. The 16 lead-time and cost-ratio cells of a dataset share the same units and alarms, so they are not independent replications; we report the range of ratios over cells and, descriptively, the number of cells whose interval excludes 1.

These intervals cover the sampling of test units given the fitted policies. They omit the variability from refitting the regressor, from the assignment of units to folds and from the selection of settings. Three results indicate its size. On PHM08, thirty repeated random 50-engine test sets with settings re-selected each time gave a standard deviation of 3.3 percentage points (Supplementary Section S9). On the batteries,

changing the time scale ($\kappa = 2$ to 6) moved the cross-validation rate by up to 3.3 points and the held-out-batch rate by up to 21.1 points. A change of the filling of one missing cycle moved per-split counts by up to three cells (Supplementary Section S5). Four leakage checks were run on PHM08: disjoint outer and inner units, causality of the features, a negative control with permuted lives, and exact reproduction of a refitted fold (Supplementary Section S4); the same four were run on every dataset of the comparison with the metric M (Supplementary Section S17). For that comparison, thirty random 80/20 splits of FD001 show the spread of an evaluation on 20 test units; because they overlap, they are summarised by their distribution, not by a pooled interval.

6. Prediction accuracy and alarm timing (Q1, Q2)

6.1. Q1: Observation time, not self-timing, separates fixed-time and alarm accuracy

A self-timed alarm is scored near failure, whereas fixed-time predictions are often scored far from it, so a comparison of the two mixes long-range prediction, near-failure estimation and alarm timing. Table 5 and Fig. 2 separate them under matched training data: every variant is fitted on the same training engines and calibrated on the same cross-fitted predictions, and one element changes at a time.

Observation time dominates. Scored at half of life, the calibrated fixed-time model placed 24.8% of its PHM08 estimates in the window; scored at 90% of life (median 21 cycles left), the same model placed 85.3%. The change is a change of the prediction problem, not of the model. At half of life the change of a pooled health signal from its early-life level (median 0.23) is comparable to its cycle-to-cycle noise (0.21), so the remaining life is only weakly identified from the data; at 20 cycles left the same signal has risen to about 1.08. Under the fit-75%/calibrate-25% protocol, the best of eleven fixed-time models at half of life, a stacked hybrid whose quantile stacker and shift were learned on the calibration engines, placed 33.5% (Fig. 2a; Supplementary Section S1). **Training allocation** mattered less: fitting on 75% of the training engines and calibrating on the other 25% gave 81.2% for the fixed-time model and 89.4% for the recipe, against 85.3% and 91.7% with cross-fitting on all of them, and neither difference was significant.

Target and features made a difference at a fixed observation time. At 90% of life the recipe's capped regressor placed 91.7% in the window against 85.3% for the fixed-time model (27 against 13 discordant engines, $p = 0.038$). At exactly 20 cycles left the figures were 89.9% against 84.4% (27 against 15 discordant engines, $p = 0.088$). **Self-timing** did not add measurable accuracy for either regressor. With the

Table 4

Policies compared, the signal they act on, the settings they select and the data and criterion used for selection. OOF: out-of-fold predictions of the training units from inner five-fold cross-validation; nothing is selected on test units.

Policy	Signal	Settings selected	Selection data and criterion	Used in
STW, window-tuned	Capped GBM estimate, smoothed and shifted	Cap c , span m , debounce k , shift s (per H); in the cost analysis also H	OOF; most correct first warnings (up to 1,476 settings per H); H by lowest training cost rate	All
Constant-estimate control	Alarm times of the window-tuned STW	One constant estimate	OOF alarms; most correct	Q2
Health-indicator alarm	Linear health indicator, smoothed	Span, debounce, threshold (120 quantiles); constant estimate (window)	OOF; most correct, or lowest training cost rate	Q2, Q3
Calibrated health-indicator alarm	Linear health indicator mapped to cycles (isotonic), smoothed and shifted	Cap c , span m , debounce k , shift s (per H)	OOF for the isotonic map and the trigger; most correct first warnings	Q2
STW, cost-tuned (plain, 1-SE, 80th percentile)	Capped GBM estimate, smoothed	c, m, k , threshold $\tau \in \{1, \dots, 60\}$ (2,160 settings)	OOF; lowest training cost rate; 1-SE: largest mean remaining life at the alarm within one bootstrap SE of the minimum; or lowest 80th bootstrap percentile	Q3
Quantile alarm (plain, 1-SE)	10% quantile GBM, smoothed	m, k, τ	OOF; as above	Q3
Capacity threshold (battery)	Block-mean discharge capacity, smoothed	Span, debounce, threshold; constant estimate (window)	Training cells; most correct, or lowest training cost rate (plain, 1-SE)	Q3, Q4
Capacity trend (battery)	Remaining life from capacity extrapolated to 0.88 Ah	Trend length, span, debounce, shift (window) or threshold τ (cost)	Training cells; most correct (window), lowest training cost rate (plain, 1-SE), H by cost for the window-tuned version	Q3, Q4
Optimal age replacement	Age	Replacement age T	Training lives; lowest cost rate	Q3
Policy 1 of Kamariotis et al. (P-cl, P-res)	$\Pr(\text{RUL} \leq \Delta T)$ at decision times: calibrated classifier, or empirical RUL distribution per bin of the cap-60 estimate	Threshold $p_{\text{thres}} = c_p/c_c$, or optimised	OOF (classifier calibration, bins); lowest training cost rate on the grid	Q3 (M)
Policy 3 of Kamariotis et al.	Binned RUL distribution of the cap-60 (or cap-125) estimate	None (cost-based decision rule)	OOF bins; age-replacement cost rate on training lives	Q3 (M)
Fixed-time GBM	Uncapped GBM estimate at a fixed observation time	Shift; with the trigger: span, debounce, shift	OOF (cross-fitted) or calibration engines (75/25)	Q1
Age-only regressor; MLP + Kalman filter	Age; MLP estimate filtered over time	As STW, window-tuned	OOF; most correct first warnings	Q4 (protocols)

Battery policies: lead times, thresholds and cycles of use in cycles, with each model working in blocks of its own κ (fully nested). Under the metric M every policy acts only at decision times $t_k = 10k$, and our alarms are mapped to the next decision time. The engine cost results of the window-tuned, cost-tuned, health-indicator and age policies were reproduced unit by unit by a separate rerun that shares the alarm and cost functions.

trigger, the fixed-time model reached 84.4%, its rate at 20 cycles left (17 against 17 discordant engines). The recipe's self-timed first warning reached 91.7%, the same rate as its regressor scored at 90% of life (8 against 8 discordant engines, $p = 1.000$). Against the calibrated fixed-time model at 90% of life the self-timed recipe was higher (26 against 12 discordant engines, $p = 0.034$), but that difference is accounted for by the regressor, not by the timing.

A second dataset. On FD004 (Table 5, last column), whose 249 engines have two fault modes and six regimes, the calibrated fixed-time model placed 18.9% of its estimates in the window at half of life and 66.7% at 90% of life. Target and features mattered more than on PHM08: at 90% of life the recipe's regressor placed

82.3% (61 against 22 discordant engines, $p < 0.001$). The self-timed recipe placed 82.7%, no more than its regressor at 90% of life (18 against 17 discordant engines, $p = 1.000$); with the trigger, the fixed-time model reached 72.3%. By the criterion fixed before this analysis (at least 30 points between half and 90% of life, and self-timing not significantly above the same regressor at 90% of life), the PHM08 conclusion repeats on FD004. One result goes against self-timing: on FD004 the regressor scored at the oracle time of 20 cycles left was more accurate than the self-timed alarm (89.2% against 82.7%; correct only at the oracle time 32 engines, only with the alarm 16, $p = 0.029$), so with two fault modes the alarm lost some accuracy relative to an observation time that cannot be known in service.

Table 5

Fixed-time and self-timed evaluation under matched training data (five-fold cross-validation; PHM08, 218 engines; FD004, 249 engines). Except in the last two rows (PHM08 only), every variant is fitted on all outer training engines, and calibration and selection use inner out-of-fold predictions of the training engines only; the last two rows fit on 75% and calibrate or tune on the other 25%. Correct: $\hat{y} - 2 \leq r \leq \hat{y} + 13$.

Variant	Observation time	Regressor	Calibration or selection	PHM08: correct (%)	95% CI	FD004: correct (%)
Fixed-time, half of life	50% of life (PHM08: median 105 cycles left)	Fixed-time GBM	Shift	54 (24.8)	19.2–31.1%	47/249 (18.9)
Fixed-time, near failure	90% of life (PHM08: median 21 left)	Fixed-time GBM	Shift	186 (85.3)	79.9–89.7%	166/249 (66.7)
Fixed-time, oracle time	20 true cycles left	Fixed-time GBM	Shift	184 (84.4)	78.9–89.0%	168/249 (67.5)
Fixed-time model, self-timed	Own alarm, $H = 20$	Fixed-time GBM	Span, debounce, shift	184 (84.4)	78.9–89.0%	180/249 (72.3)
Recipe regressor, near failure	90% of life	Capped GBM (recipe)	Cap, shift	200 (91.7)	87.3–95.0%	205/249 (82.3)
Recipe regressor, oracle time	20 true cycles left	Capped GBM (recipe)	Cap, shift	196 (89.9)	85.1–93.6%	222/249 (89.2)
Recipe, self-timed	Own alarm, $H = 20$	Capped GBM (recipe)	Cap, span, debounce, shift	200 (91.7)	87.3–95.0%	206/249 (82.7)
Fit 75% / calibrate 25%: fixed-time	90% of life	Fixed-time GBM	Shift on 25%	177 (81.2)	75.4–86.2%	–
Fit 75% / tune 25%: recipe	Own alarm, $H = 20$	Capped GBM (recipe)	All settings on 25%	195 (89.4)	84.6–93.2%	–

Fixed-time GBM: no target cap, features of seven temperature and speed sensors. Oracle time: uses the true life and cannot be deployed. Paired exact McNemar tests (only first / only second correct): self-timed vs 90% of life, recipe 8/8 ($p = 1.000$), fixed-time model 16/18 ($p = 0.864$); recipe vs fixed-time regressor at 90% of life 27/13 ($p = 0.038$), at 20 cycles left 27/15 ($p = 0.088$); cross-fitted vs 75/25, fixed-time 18/9 ($p = 0.122$), recipe 9/4 ($p = 0.267$).

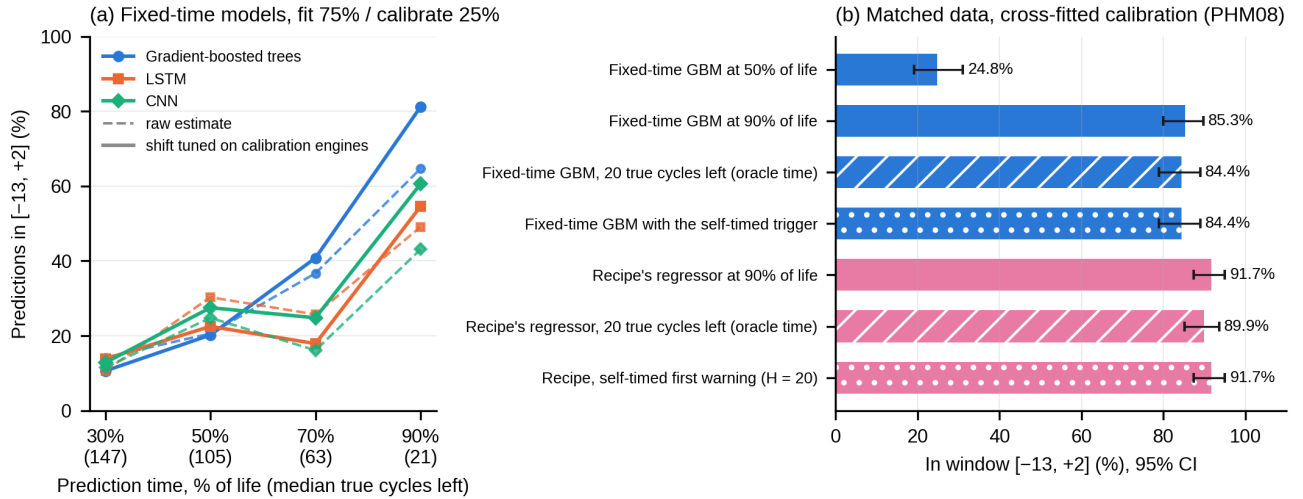


Fig. 2: Fixed-time and self-timed evaluation on PHM08 (218 engines, five-fold cross-validation), window $[-13, +2]$. (a) Fixed-time models fitted on 75% of the outer training engines and scored at fractions of each engine's life, on their raw estimates (dashed) and with a shift chosen on the other 25% (solid); the median true remaining life at each fraction is in brackets. Accuracy rises with the fraction of life for every model. (b) Matched comparison (Table 5): all variants fitted on all outer training engines with cross-fitted calibration; hatched: oracle observation time (//) or self-timed alarm (dots). Error bars: 95% Clopper–Pearson intervals.

Answer to Q1. On PHM08 the contrast between long-range fixed-time prediction (24.8% at half of life under cross-fitting; 33.5% for the best of eleven models under the fit-75%/calibrate-25% protocol, against 89.4% for the recipe under that protocol) and the self-timed alarm (91.7%) is mostly a contrast of observation time, with a further, smaller contribution from target and feature design, which was larger on FD004. A study that compares a fixed-time model scored at a fraction of life with an alarm scored at its own time therefore compares two prediction problems of different difficulty. What self-timing contributes is not accuracy but the ability to act without knowing the life: a fraction of life cannot be observed in service, whereas the alarm reaches near-failure accuracy from the history alone, with no missed engine and a median of 19 cycles of notice on PHM08. Whether such an alarm is a good maintenance policy is a separate question (Section 7). These are paired comparisons on 218 and 249 engines of one simulator family, and differences of a few points could go undetected.

6.2. Q2: The window score rewards alarm timing

The scores look high. Table 6 gives the first-warning results on the engines. At $H = 20$, 200 of 218 PHM08 engines (91.7%, 95% CI 87.3–95.0%) received a correct first warning; 8 were late, 10 too early and none was missed. The alarm came a median of 19 cycles before failure (5–95%: 13–28), and 95.4% of alarms were raised with at least 13 cycles left, so the early limit was active. On C-MAPSS the rate was 88.0% on FD001, 94.6% on FD002, 96.0% on FD003 and 82.7% on FD004, which has two fault modes and six regimes; 6 of its 26 too-early alarms came more than 30 cycles early. Pooled over the 927 engines it was 90.2%. The question is what these numbers measure.

A constant estimate scores like the model. The constant-estimate control keeps the recipe's alarm times and discards its estimate, replacing it by one constant per fold chosen on training alarms. If the window score measured the quality of the remaining-life estimate, this control should score clearly lower. It did not (Table 7). Over the five engine datasets the constant placed 90.8% of first warnings in the window, against 90.2% with the model's estimate (27 against 33 discordant engines, $p = 0.519$). On each of the seven datasets on which it was run, the constant scored between -2.4 and $+3.6$ percentage points relative to the model's estimate: lower on FD002, FD003, Battery, higher on FD001, FD004, N-CMAPSS, equal on PHM08. No difference was significant; no equivalence margin was tested, so the result bounds the contribution of the estimate rather than proving it zero. The reason is visible in the alarms. Because the alarm is raised when the smoothed, shifted estimate first stays below H , the reported estimate varies little between units:

on PHM08 at $H = 20$ its standard deviation at the alarm was 2.1 cycles (5–95%: 11.2–18.2), against 4.8 cycles for the true remaining life. Correctness, $\hat{y} - 2 \leq r \leq \hat{y} + 13$, therefore depends mainly on where the alarm falls: for $\hat{y}$ between 11.2 and 18.2 the accepted range of r runs from at least 9.2 to at most 31.2 cycles. A high window score for a threshold-crossing alarm is evidence of well-timed alarms, not, by itself, of an accurate remaining-life estimate.

Timing quality is not trivial. That the score rewards timing does not make every timing mechanism equally good. The health-indicator threshold alarm, which times its alarm on a classical linear health indicator, placed 49.4–57.0% of first warnings in the window, significantly fewer than the recipe on each of the seven datasets after Holm's correction (largest adjusted $p = 0.020$, N-CMAPSS), with many alarms too early (64 of 218 on PHM08). A fairer indicator-based opponent maps the same indicator to cycles to failure and uses the recipe's own trigger and selection rule. On PHM08 and FD001–FD004 this calibrated indicator placed 48.0–55.8% of first warnings in the window, close to the uncalibrated alarm and significantly fewer than the recipe on each (largest Holm-adjusted $p < 0.001$). On N-CMAPSS it placed 69.7%, against 68.7% for the recipe (19 against 20 discordant engines, Holm-adjusted $p = 1.000$) and 53.5% for the uncalibrated alarm. By the interpretation fixed before this analysis, on PHM08 and FD001–FD004 the gradient-boosted regressor contributes timing information that a linear indicator expressed in cycles does not, whereas on N-CMAPSS, where the recipe itself was weak, calibration in cycles accounts for the recipe's timing. The regressor's value therefore lies in when its trajectory crosses the level, which the window score rewards, rather than in the number it reports at the crossing, which the score barely uses. Under an identical protocol on PHM08 the recipe scored significantly higher than re-implemented LSTM, similarity and convolutional approaches, and not significantly differently from a multilayer perceptron with a Kalman filter that shares its features or from a hybrid Transformer given its inputs (Supplementary Section S15), so it is a reasonable instrument for this analysis, not a claim about the best model.

Tuning to the window acts on the timing. Tuning selects, among other settings, the shift s added to the smoothed estimate, and Proposition 1 describes its effect for a fixed error distribution. We varied the shift on the test units of each fold by up to ± 4 around the value selected on training data at $H = 20$, keeping the other settings, and recorded the alarm cycle, the true remaining life at the alarm and the unshifted error ε (Fig. 3). On each of the seven datasets the shift acted mainly by moving the alarm. Each unit of shift changed the median true remaining life at the alarm by -1.0 to -0.9 cycles (blocks for the battery), against -0.1 to 0.1 for the median of ε . Assumption

Table 6

First-warning results of the recipe on the development and additional evaluation datasets (five-fold cross-validation by engine, nested selection of the settings). Correct: $\hat{y} - 2 \leq r \leq \hat{y} + 13$ at the alarm. The last two columns show the notice actually given, which the in-window rate does not.

Dataset	H	Correct	%	95% CI	Late or missed	Too early	True cycles left at alarm, median (5–95%)	Alarms with ≥ 13 cycles left (%)
PHM08	10	210/218	96.3	92.9–98.4	4	4	10 (5–17)	21.7
	15	205/218	94.0	90.0–96.8	6	7	15 (8–22)	65.6
	20	200/218	91.7	87.3–95.0	8	10	19 (13–28)	95.4
	30	182/218	83.5	77.9–88.2	13	23	30 (20–43)	100.0
	40	173/218	79.4	73.4–84.5	22	23	38.5 (28–51)	100.0
FD001	10	98/100	98.0	93.0–99.8	2	0	11 (7–17)	32.0
	15	95/100	95.0	88.7–98.4	4	1	14 (8–21)	59.0
	20	88/100	88.0	80.0–93.6	4	8	19 (13–29)	96.0
FD002	10	250/260	96.2	93.0–98.1	5	5	11 (5–17)	32.7
	15	252/260	96.9	94.0–98.7	4	4	15 (9–22)	73.1
	20	246/260	94.6	91.1–97.0	7	7	19 (12–26)	93.8
FD003	10	94/100	94.0	87.4–97.8	5	1	11 (5–17)	32.7
	15	94/100	94.0	87.4–97.8	4	2	16 (8–24)	77.8
	20	96/100	96.0	90.1–98.9	1	3	21 (16–29)	98.0
FD004	10	225/249	90.4	86.0–93.7	7	17	10 (4–20)	33.3
	15	214/249	85.9	81.0–90.0	18	17	14 (6–26)	61.8
	20	206/249	82.7	77.5–87.2	17	26	20 (12–34)	94.4

Pooled over PHM08 and FD001–FD004: 877/927 (94.6%) at $H = 10$, 860/927 (92.8%) at $H = 15$, 836/927 (90.2%) at $H = 20$. The five datasets come from one simulator, so the pooled rate is not an estimate for unrelated systems.

Table 7

Timing controls at $H = 20$ (five-fold cross-validation): the recipe, the same alarm times with one constant estimate per fold, a health-indicator threshold alarm without a remaining-life regressor, and the same indicator calibrated to cycles and put through the recipe's trigger.

Dataset	Recipe: correct (%)	Constant estimate: correct (%)	Discordant (recipe only / control only); p	Health-indicator alarm: correct (%)	Discordant; p (Holm)	Calibrated indicator: correct (%)	Discordant; p (Holm)
PHM08	200/218 (91.7)	200 (91.7)	7 / 7; 1.000	122 (56.0)	83 / 5; < 0.001	116 (53.2)	91 / 7; < 0.001
FD001	88/100 (88.0)	90 (90.0)	2 / 4; 0.688	51 (51.0)	41 / 4; < 0.001	48 (48.0)	44 / 4; < 0.001
FD002	246/260 (94.6)	243 (93.5)	9 / 6; 0.607	142 (54.6)	105 / 1; < 0.001	145 (55.8)	104 / 3; < 0.001
FD003	96/100 (96.0)	94 (94.0)	4 / 2; 0.688	57 (57.0)	40 / 1; < 0.001	54 (54.0)	43 / 1; < 0.001
FD004	206/249 (82.7)	215 (86.3)	5 / 14; 0.064	123 (49.4)	94 / 11; < 0.001	121 (48.6)	97 / 12; < 0.001
N-CMAPSS Battery	68/99 (68.7) 118/123 (95.9)	70 (70.7) 115 (93.5)	7 / 9; 0.804 7 / 4; 0.549	53 (53.5) 67 (54.5)	26 / 11; 0.020 54 / 3; < 0.001	69 (69.7) –	19 / 20; 1.000 –
Engines pooled (PHM08, FD001–FD004)	836/927 (90.2)	842 (90.8)	27 / 33; 0.519	495 (53.4)	–	484 (52.2)	–

Exact McNemar tests. Constant-estimate column: unadjusted p ; health-indicator columns: Holm-adjusted over the seven (calibrated: six engine) datasets. Battery: task in blocks with κ from the outer training cells (convention B); the calibrated indicator was not run on the battery, for which the capacity baselines of Section 8 serve as domain baselines (Supplementary Section S10 gives every level).

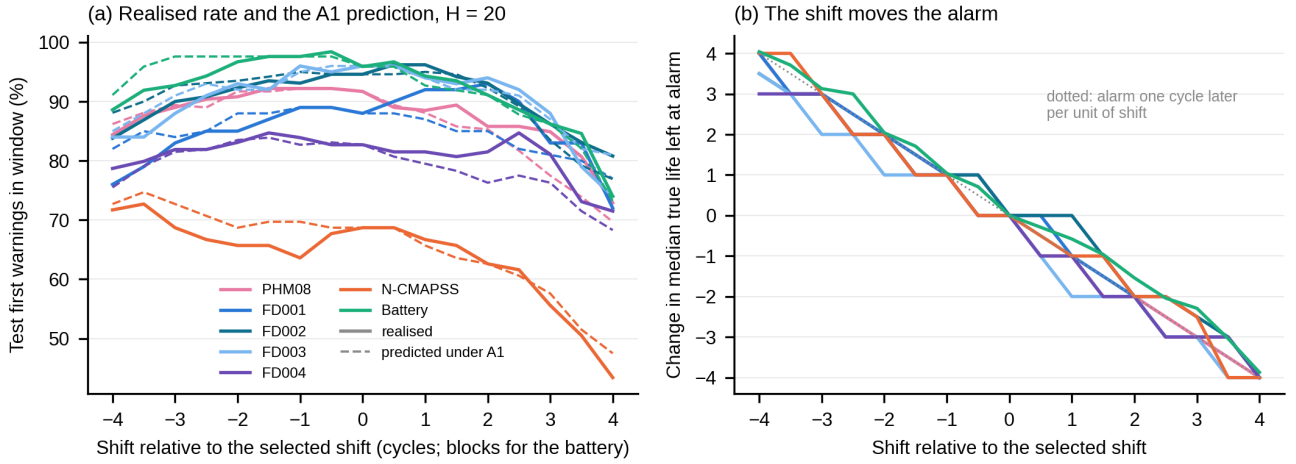


Fig. 3: What tuning the shift acts on: test units at $H = 20$ with the other settings fixed. (a) Realised in-window rate (solid) and the rate predicted by Proposition 1 from the errors at the selected shift (dashed) as the shift moves away from the value selected on training data. (b) Change of the median true remaining life at the alarm relative to the selected shift; dotted: one cycle less per unit of shift. Battery in blocks of κ from the outer training cells (convention B).

A1 is therefore violated by construction, yet the error at the alarm changed slowly. Within ± 2 of the selected shift, the largest difference between the realised rate and the rate predicted by Proposition 1 from the errors at the selected shift was 1.6 to 8.0 percentage points, depending on the dataset (largest on FD001); within ± 4 it was 4.2 to 8.0 points. The proposition thus describes the direction and rough size of the response, with errors of several points; it is not a bound for the sequential policy. In a component ablation on PHM08, removing the shift caused the largest loss among the trigger and target components (Supplementary Section S15).

What the score conceals. A correct alarm with $H = 20$ means a remaining life of roughly 9 to 31 cycles; the score does not distinguish 10 from 30 cycles of notice, nor say whether the notice suffices for a given lead time (Fig. 5). It treats every error inside $[-13, +2]$ alike and every error outside it alike, whatever its size, and it is silent about cost. The warning level trades score for notice: on PHM08, lower levels gave higher rates but less notice (96.3% at $H = 10$), and $H = 30$ and 40 gave 83.5% and 79.4%; on FD002 and FD003 the rate did not fall monotonically with H (Fig. 4). Section 8.1 shows, on the battery cells, a case in which the policy with the higher window score gave much less notice, and Section 7 shows what the score leaves out about cost.

Answer to Q2. For a threshold-crossing alarm, the asymmetric window score is driven mainly by when the alarm fires. The reported estimate added little that a constant could not supply, and tuning to the window acted mainly on the alarm time. The timing itself depended on the regressor, so the score does separate good from poor timing mechanisms; what it does not

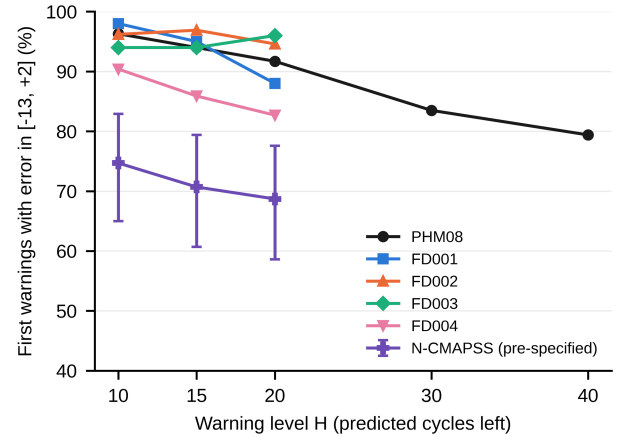


Fig. 4: Share of first warnings in the window as a function of the warning level H (five-fold cross-validation). A higher level asks for more notice. Error bars: 95% Clopper-Pearson intervals for the pre-specified N-CMAPSS analysis.

measure is the accuracy of the remaining-life estimate, the notice given or the cost incurred.

7. Maintenance-policy value (Q3)

A score is a useful surrogate for maintenance only if optimising it produces policies with good decision outcomes. We test this in two ways. Section 7.1 prices the alarms in the renewal-reward model of Section 3.3 and compares window tuning with age replacement and with tuning the same regressor directly to cost. Sections 7.2–7.5 evaluate the alarms under the decision-oriented metric M of Kamariotis et al. [9], with their decision grid, against their probability-threshold policies built on the same regressor.

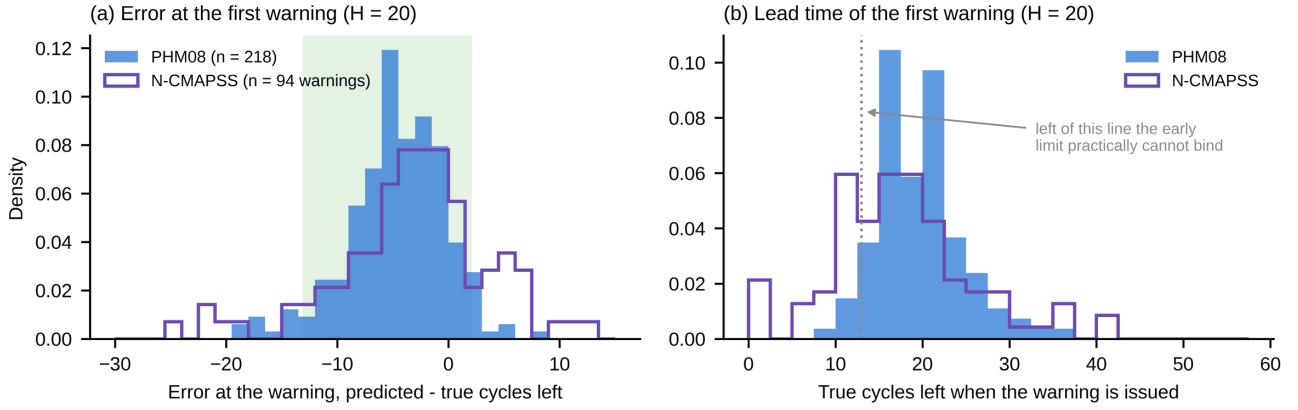


Fig. 5: Distribution of (a) the warning error and (b) the true remaining life at the alarm at $H = 20$ for PHM08 (five-fold cross-validation) and N-CMAPSS (pre-specified analysis). Shaded: the window; dotted: 13 cycles, below which the early limit practically cannot bind. Alarms with the same window outcome can give very different notice.

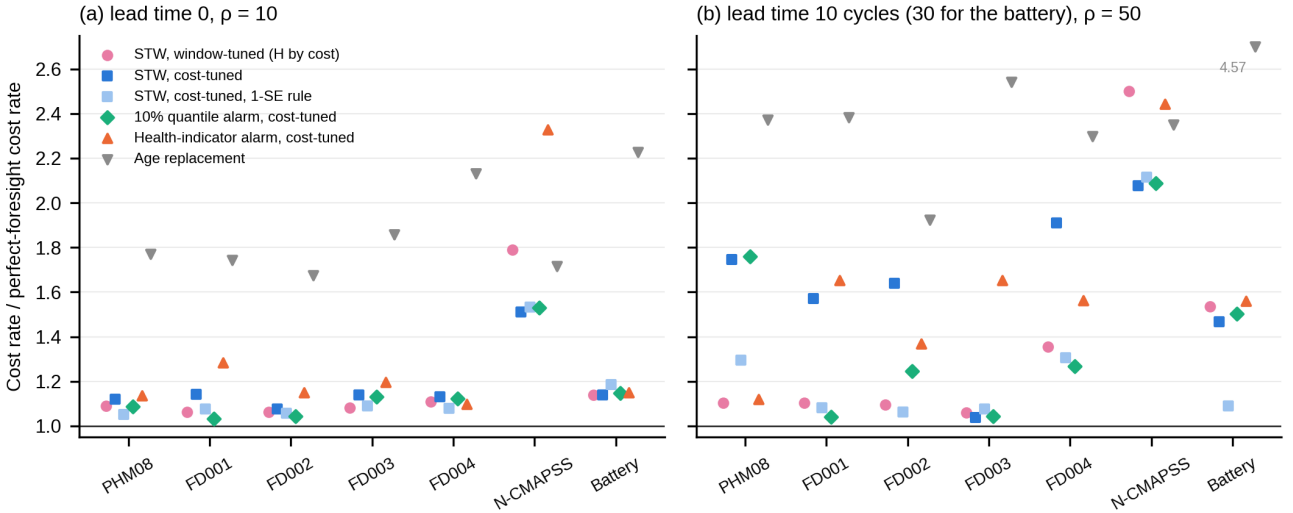


Fig. 6: Long-run cost rate relative to perfect foresight (1.00 = replacement one cycle before every failure) for policies tuned on training units: (a) without lead time and $\rho = 10$; (b) with a lead time of 10 cycles (30 cycles for the battery) and $\rho = 50$. Five-fold cross-validation; battery fully nested, with the task in cycles (convention C). Off-scale values are labelled.

7.1. Window tuning, age replacement and direct cost tuning

Table 8 and Fig. 6 compare the policies under identical splits, information, lead times and cost ratios. Against optimal age replacement, the window-tuned alarms were cheaper with a paired interval excluding equality in 16, 16, 15, 16, 15, 16 of 16 cells on PHM08, FD001, FD002, FD003, FD004, Battery, respectively. Their cost rate was 1.07–1.28 times that of perfect foresight on PHM08, against 1.70–2.37 for age replacement. N-CMAPSS was the exception: in no cell did the interval exclude equality, and the point estimate favoured age replacement in 4 of 16 cells.

Against policies tuned directly to cost the results depend on the dataset (Table 8). Plain cost tuning of the same regressor was significantly cheaper than window tuning in 4, 8, 9 and 13 of the 16 cells on

FD001, FD003, N-CMAPSS and the batteries (none on PHM08, FD002 and FD004), and significantly more expensive in no cell on any dataset. Its occasional large losses remain. On PHM08 with $\ell = 10$ and $\rho = 50$ it cost 1.75 times perfect foresight, against 1.10 for window tuning, with 3 test failures against 0. The one-standard-error rule reduced that loss to 1.30, still above window tuning there. Across cells it was significantly cheaper than window tuning in 4, 7, 11, 3, 9 and 2 of the 16 cells on PHM08, FD001, FD002, FD004, N-CMAPSS and the batteries (none on FD003), and significantly more expensive in 1, 4, 1, 16 and 2 of the 16 cells on PHM08, FD001, FD002, FD003 and the batteries (none on FD004 and N-CMAPSS). The cost-tuned 10% quantile alarm was significantly cheaper than window tuning in 3, 16, 4, 4, 3, 6 and 10 of the 16 cells on PHM08, FD001, FD002,

Table 8

Cost of the window-tuned alarms relative to each alternative policy, by dataset. Each entry: median (minimum–maximum) over the 16 lead-time and cost-ratio cells of the ratio of cost rates (window-tuned / alternative; below 1 favours the window-tuned policy); then the number of cells in which the paired 95% bootstrap interval lies entirely below 1 / entirely above 1.

Dataset	Age replacement	Health-indicator alarm	STW, cost-tuned	STW, cost-tuned, 1-SE	STW, cost-tuned, 80th pct.	Quantile alarm, cost-tuned	Quantile alarm, 1-SE
PHM08	0.60 (0.45–0.64); 16/0	0.96 (0.79–0.99); 10/0	0.93 (0.61–0.99); 0/0	1.01 (0.84–1.22); 1/4	0.93 (0.61–0.99); 0/0	1.00 (0.63–1.05); 0/3	1.00 (0.82–1.21); 0/2
FD001	0.59 (0.45–0.65); 16/0	0.83 (0.49–0.95); 9/0	0.96 (0.68–1.03); 0/4	1.00 (0.99–1.02); 4/7	0.96 (0.68–1.03); 0/4	1.04 (1.03–1.06); 0/16	1.02 (1.01–1.04); 0/16
FD002	0.65 (0.55–0.69); 15/0	0.94 (0.80–1.16); 12/0	0.97 (0.66–1.06); 0/0	1.01 (0.85–1.05); 1/11	1.00 (0.66–1.06); 0/4	1.01 (0.74–1.06); 0/4	1.02 (0.99–1.24); 1/14
FD003	0.55 (0.42–0.61); 16/0	0.85 (0.63–0.95); 16/0	1.01 (0.51–1.03); 0/8	0.98 (0.98–0.99); 16/0	1.01 (0.51–1.03); 0/8	0.90 (0.51–1.02); 0/4	0.98 (0.97–1.00); 12/3
FD004	0.56 (0.48–0.67); 15/0	0.97 (0.84–1.17); 2/0	0.97 (0.71–1.01); 0/0	1.00 (0.86–1.19); 0/3	0.97 (0.71–1.01); 0/0	1.02 (0.76–1.20); 0/3	1.04 (1.01–1.41); 0/11
N-CMAPSS	0.98 (0.69–1.06); 0/0	0.80 (0.44–1.05); 6/0	1.17 (0.79–1.73); 0/9	1.14 (0.78–1.73); 0/9	1.16 (0.79–1.73); 0/6	1.12 (0.78–1.76); 0/6	1.11 (0.77–1.73); 0/4
Battery	0.49 (0.33–0.52); 16/0	0.98 (0.91–1.04); 11/4	1.04 (1.00–1.05); 0/13	1.04 (0.94–1.41); 2/2	1.04 (0.99–1.05); 1/13	1.02 (0.98–1.04); 2/10	0.99 (0.93–1.37); 6/0

Five-fold cross-validation; lead times $\ell \in \{0, 5, 10, 15\}$ cycles (battery 0, 15, 30, 45 cycles) and failure-cost ratios $\rho \in \{5, 10, 20, 50\}$. The 16 cells of a dataset share units and alarms and are not independent; the counts are descriptive. Battery: cross-validation by cell, fully nested preprocessing with the task in cycles (convention C); the battery policies under a held-out batch, including the capacity trend, are in Table 12. The cost rates relative to perfect foresight for every cell are in Supplementary Section S11.

FD003, FD004, N-CMAPSS and the batteries, and more expensive in 2 of the 16 cells on the batteries (none elsewhere). The health-indicator alarm was usually the more expensive: window tuning was significantly cheaper in 10, 9, 12, 16, 2, 6 and 11 of the 16 cells on PHM08, FD001, FD002, FD003, FD004, N-CMAPSS and the batteries, and more expensive in 4 of the 16 cells on the batteries (none elsewhere). On the batteries, the cost-tuned capacity-threshold policy was significantly cheaper than window tuning in 4 cells and never significantly more expensive.

Relative to the cheapest condition-based policy in each cell, the mean excess cost over the 112 cells was 8.9% for window tuning, 14.5% for plain cost tuning, 4.8% with the one-standard-error rule, 11.1% for the quantile alarm (5.5% with the one-standard-error rule), 27.4% for the health-indicator alarm and 88.2% for age replacement. These means are descriptive, because the cells are correlated. The policy with the lowest mean excess cost differed between datasets: window-tuned STW on PHM08, cost-tuned quantile alarm on FD001, quantile alarm with the 1-SE rule on FD002, window-tuned STW on FD003, quantile alarm with the 1-SE rule on FD004, cost-tuned STW (80th percentile) on N-CMAPSS, cost-tuned STW with the 1-SE rule on Battery. The largest single excess cost was 94.8% for plain cost tuning (FD003), 75.7% for window tuning (N-CMAPSS, $\ell = 5$, $\rho = 50$), 52.6% with the one-standard-error rule (Battery) and 102.1%

for the quantile alarm (N-CMAPSS). Window tuning thus avoided the largest losses of plain cost tuning on several datasets, plausibly through the margin that the narrow late side of the window builds in, but that mechanism was not tested and it did not hold on N-CMAPSS. No tuning rule was cheapest on every dataset, and window tuning is not a reliable surrogate for cost optimisation.

7.2. The decision-oriented metric M and its relation to our cost ratio

Kamariotis et al. [9] assess a prognostic algorithm through the maintenance decisions that a policy takes with its predictions. Decisions can be taken only at times $t_k = k\Delta T$; at each one the policy replaces the component preventively, at cost c_p , or lets it run; a component that fails first is replaced correctively at cost $c_c > c_p$. With C_i the cost and T_i the length of the life cycle of test unit i , the long-run cost rate is estimated by $R = \Sigma C_i / \Sigma T_i$, and the metric is its relative excess over a perfect policy that replaces every unit at the last decision time before its failure,

$$M = (R - R_{\text{perf}}) / R_{\text{perf}}. \quad (7)$$

They use M to compare algorithms, to choose policy thresholds on training units and to tune hyperparameters, on FD001 split into 80 training and 20 test engines with $\Delta T = 10$ cycles, and they state that an evaluation

of M on 20 units is subject to significant statistical uncertainty. The status of our comparison is as follows. **Re-implemented:** the metric with its first-order variance (their Eqs. 2–9), the grid-perfect reference, and their policies 1 and 3, applied on FD001–FD004 to our regressor under their conventions. **Reproduced:** only the qualitative findings of their simulation study, with our implementation on their simulator (Section 7.3). **Assumed:** a construction of the failure probability from our point regressor, the mapping of our alarms to decision times, and a grid of cost ratios (below). **Not verifiable:** their code, data split and prognostic models are not available to us, so we do not compare with the values they report, which come from other models on one 20-engine split, and the published version of their article should be checked against the preprint used here. Everything in this section is exploratory; the analysis plans, including seeds and the definition of a win, were hashed before the runs (Table 3).

With $\ell = 0$ and $\rho = c_c/c_p$, our cost rate relative to perfect foresight (Section 3.3) and M differ only in the reference policy. Ours replaces at $L - 1$; theirs at the last decision time before L . Hence the ratio of Section 3.3 equals $(1 + M)(1 + g)$, where $g = \Sigma(L_i - 1) / \Sigma T_i^{\text{perf}} - 1$ is a property of the test lives and the grid alone. The ratio equals $1 + M$ only for $\Delta T = 1$. With $\Delta T = 10$, g was 1.7–2.3% on PHM08 and C-MAPSS and 7.3% on N-CMAPSS, whose lives of 48–100 flight cycles make a 10-cycle grid coarse; we therefore report no M for N-CMAPSS. Three further differences matter. (1) **Decision grid.** Our alarms can fire at any cycle. On their grid an alarm acts at the next decision time, and an alarm whose decision time is at or beyond the failure counts as a failure. Without this mapping M can be negative: in the earlier battery cost analysis (convention B) with $\Delta T = 30$ cycles, the unmapped capacity-threshold policy reached $M = -0.37\%$, because its alarms fell between decision times. Our policies were therefore re-selected on training units under the grid-mapped cost. (2) **Lead time.** Their replacement-only setting has no lead time, so the comparison uses $\ell = 0$ only; their ordering setting, which has a different lead-time model, was not implemented. (3) **Cost ratios.** We use $c_p/c_c \in \{0.01, 0.02, 0.05, 0.1, 0.2, 0.5, 0.8\}$, chosen to span the ranges shown in their figures; 0.1, the only ratio for which their text reports a value, is primary and the others are descriptive.

7.3. Their policies on our regressor

Their policy 1 replaces at the first decision time at which $\Pr(\text{RUL} \leq \Delta T) \geq p_{\text{thres}}$, with $p_{\text{thres}} = c_p/c_c$ or a threshold optimised on training units; their policy 3 replaces when the replacement time that minimises the expected cost under the predictive RUL distribution, with the remaining life valued at an age-replacement cost rate, falls before the next decision time (their Eq. 14, option 1). Both need a failure probability,

which our point regressor does not give. We built it in two ways from training units only: **P-cls**, a gradient-boosted classifier of $\text{RUL} \leq 10$ on the recipe's features, calibrated by isotonic regression on inner out-of-fold predictions; and **P-res**, the empirical distribution of the true RUL in equal-count bins of the inner out-of-fold estimates of the cap-60 regressor (at least 200 rows per bin, monotonicity enforced, RUL above the cap set to the cap; cap 125 as a sensitivity analysis for policy 3). Differences between their policies and ours therefore reflect the probability model as much as the policy. Optimised thresholds were chosen on the inner out-of-fold predictions; when several thresholds tied, the middle of the tied range was used, and the largest tied threshold, which corresponds to the choice shown in their Fig. 9a, was computed as well. A policy is said to beat another only when the paired 95% bootstrap interval of the difference in M (2,000 resamples of test units, the grid-perfect reference recomputed in each) excludes zero in the five-fold run, which tests each unit once. On their simulator (their Section 3) our implementation reproduced their qualitative findings at all seven ratios: the optimised policy 1 had the lowest M and policy 3 was below policy 2. There, as in their study, the threshold was optimised and scored on the same simulated units, so that agreement partly reflects in-sample optimisation.

7.4. Results under M

FD001. At $c_p/c_c = 0.1$ (Table 9), their policy 1 with P-res reached $M = 5.69\%$ without a failure, below our 1-SE cost-tuned alarm (7.04%) and our window-tuned alarm (8.08%); in both comparisons the paired interval excluded zero (Table 10), so on FD001 **their policy beats ours**. It did so at 5 of the seven ratios against the window-tuned alarm and 6 against the 1-SE alarm, and ours never beat it. Our alarms beat their policy 3 (10.34%) and age replacement (79.25%). Against policy 1 with P-cls (12.02%, 1 failure) the comparison was unclear at every ratio: one failure among 100 units widens the interval to tens of points. Two observations explain the result. First, the grid penalises an alarm that was not designed for it: our window-tuned alarm, scored at its own cycle, reached 3.93%, but such an alarm is not admissible on their grid, and mapped to it, with settings re-selected for the grid, it reached 8.08%. Second, with bins about 10 cycles wide near failure, P-res policy 1 amounts to replacing at the first decision time at which the regressor's estimate falls below about 21.5 cycles: the same regressor, used through a simple grid-aware rule, gave lower M than through our trigger. The interval of their Eq. 7 treats R_{perf} as fixed. It was ± 4.58 points even for the perfect policy, whose M is 0 by definition, which is why our verdicts use the paired bootstrap.

Table 9

Metric M (%) under the conventions of Kamariotis et al. ($\Delta T = 10$ cycles, no lead time, $c_p/c_c = 0.1$): five-fold cross-validation by engine, every unit tested once. FD001: M with its paired-bootstrap 95% interval, the interval of their Eq. 7, and failures; FD002–FD004: M and, in brackets, failures.

Policy	FD001: M (bootstrap 95% CI)	FD001: Eq. 7 interval	FD001: failures	FD002	FD003	FD004
Their policy 1, P-res, optimised	5.69 (5.17–6.24)	0.72 to 10.66	0	5.66 (0)	7.88 (0)	11.25 (1)
Their policy 1, P-res, $p = c_p/c_c$	5.69 (5.17–6.24)	0.72 to 10.66	0	5.66 (0)	46.79 (5)	64.22 (17)
Their policy 1, P-clc, optimised	12.02 (2.41–30.69)	–7.26 to 31.29	1	7.11 (1)	3.46 (0)	12.83 (2)
Their policy 1, P-clc, $p = c_p/c_c$	11.22 (1.71–29.77)	–7.95 to 30.39	1	16.42 (4)	29.75 (3)	32.74 (8)
Their policy 3, P-res (cap 60)	10.34 (9.46–11.29)	4.90 to 15.79	0	5.66 (0)	10.39 (0)	15.93 (1)
Their policy 3, P-res (cap 125)	9.98 (9.15–10.80)	4.77 to 15.19	0	8.08 (0)	11.87 (0)	12.20 (1)
STW, window-tuned (mapped)	8.08 (7.21–8.97)	3.08 to 13.09	0	4.85 (0)	5.12 (0)	8.44 (0)
STW, cost-tuned, 1-SE (mapped)	7.04 (6.32–7.79)	2.04 to 12.04	0	4.91 (0)	7.26 (0)	11.63 (1)
STW, cost-tuned, plain (mapped)	13.66 (3.90–32.66)	–4.63 to 31.95	1	17.21 (4)	31.13 (3)	13.12 (2)
Health-indicator alarm, cost-tuned (mapped)	18.32 (7.73–38.48)	–1.81 to 38.45	1	14.69 (1)	18.55 (1)	13.64 (1)
Age replacement on the grid	79.25 (59.21–112.79)	50.40 to 108.11	1	73.50 (2)	73.00 (0)	91.65 (1)
STW, window-tuned, not mapped (inadmissible)	3.93 (3.45–4.41)	–0.87 to 8.74	0	3.78 (0)	6.52 (0)	8.79 (1)
Grid-perfect policy	0.00 (0.00–0.00)	–4.58 to 4.58	0	0.00 (0)	0.00 (0)	0.00 (0)

Units: FD001 100, FD002 260, FD003 100, FD004 249. P-res, P-clc: failure probability from the empirical RUL distribution per bin of the regressor's out-of-fold estimates, or from a calibrated classifier (Section 7.3). Mapped: alarms act at the next decision time. Every value of every policy at all seven cost ratios is in Supplementary Section S17.

Table 10

Paired differences in M (percentage points; our policy minus theirs; negative favours ours) at $c_p/c_c = 0.1$, with paired-bootstrap 95% intervals and the verdict (“ours” or “theirs” only when the interval excludes zero); last column: verdicts over the seven cost ratios and four datasets.

Comparison	FD001	FD002	FD003	FD004	All 28 dataset–ratio cells: ours / theirs / unclear
Window-tuned – policy 1 (P-res)	+2.39 (+1.51, +3.31); theirs	–0.80 (–1.13, –0.50); ours	–2.76 (–3.45, –2.08); ours	–2.81 (–10.83, +1.39); unclear	11 / 5 / 12
1-SE – policy 1 (P-res)	+1.35 (+0.74, +2.01); theirs	–0.74 (–1.06, –0.43); ours	–0.62 (–1.30, +0.09); unclear	+0.38 (+0.04, +0.71); theirs	9 / 10 / 9
Window-tuned – policy 1 (P-clc)	–3.93 (–22.91, +6.01); unclear	–2.26 (–9.56, +1.61); unclear	+1.66 (+1.10, +2.26); theirs	–4.39 (–15.78, +3.41); unclear	0 / 6 / 22
1-SE – policy 1 (P-clc)	–4.97 (–23.91, +4.91); unclear	–2.19 (–9.46, +1.66); unclear	+3.80 (+3.19, +4.49); theirs	–1.19 (–9.01, +2.90); unclear	0 / 12 / 16
Window-tuned – policy 3	–2.26 (–3.18, –1.42); ours	–0.80 (–1.13, –0.50); ours	–5.27 (–6.13, –4.47); ours	–7.49 (–15.97, –3.02); ours	23 / 0 / 5
Window-tuned – age replacement	–71.17 (–104.79, –51.08); ours	–68.65 (–85.91, –55.59); ours	–67.88 (–80.85, –55.97); ours	–83.21 (–100.03, –71.67); ours	28 / 0 / 0

2,000 paired bootstrap resamples of test units (seed 2027), with the grid-perfect reference recomputed in each resample; five-fold cross-validation. The seven ratios share units and alarms, so the counts in the last column are descriptive.

FD002–FD004 and the spread over splits. At the primary ratio, policy 1 with P-res beat the window-tuned alarm on FD001, the difference was unclear on FD004, and the window-tuned alarm beat it on FD002 and FD003. Against the 1-SE alarm it won on FD001 and FD004, was unclear on FD003 and lost on FD002. Our window-tuned alarm beat policy 3 on FD001, FD002, FD003 and FD004 and age replacement on all four datasets. Against policy 1 with P-clc it never won, lost on FD003 and was otherwise unclear. The lowest M at this ratio was reached by their policy 1 (P-res) on FD001 (5.69%), our window-tuned alarm on FD002 (4.85%), their policy 1 (P-clc) on FD003 (3.46%) and our window-tuned alarm on FD004 (8.44%) (Table 9),

so neither family of policies was best everywhere, and on FD004 no difference between our window-tuned alarm and their policy 1 was clear. On thirty random 80/20 splits of FD001, which overlap and are therefore summarised without a pooled interval (Supplementary Section S17), the median M at 0.1 was 3.62% for policy 1 with P-clc, 5.88% with P-res, 5.32% for the 1-SE alarm and 8.92% for the window-tuned alarm. A failure among the 20 test engines occurred in 3, 1, 3 and 3 of the 30 splits, respectively, and raised M in such a split to as much as 92.4%, which illustrates the uncertainty of a single 20-unit evaluation that Kamariotis et al. point out.

7.5. Robustness of direct threshold optimisation

Their Fig. 9a shows the risk of optimising p_{thres} when failures are expensive: the training optimum lies at the edge of a plateau, and a slightly larger threshold lets a unit or two fail and makes M very large. The same shape appeared on our data (Fig. 7b; FD001, fold 1, $c_p/c_c = 0.01$, P-cls, chosen before the run). Every threshold from 0.026 to 0.032 gave the minimal training M of 3.02%; the next threshold, 0.0328, let 1 of the 80 training engines fail and raised the training M to 129.9%. With P-res the step was larger (11 training failures and $M = 1362.4\%$ just above the plateau), because its probabilities take only a few values. The largest tied threshold, the choice their Fig. 9a illustrates, therefore sat one grid step from the training cliff; the middle of the tie sat further from it. On the test engines of that fold neither choice produced a failure.

Across test sets the risk materialised unevenly (Fig. 7c). At $c_p/c_c \leq 0.1$, the five folds of FD001–FD004 give 80 fold–ratio cells for each policy. Policy 1 with an optimised threshold had at least one test failure in 12 of them with P-cls and 4 with P-res, against 19 and 11 with $p_{\text{thres}} = c_p/c_c$; plain cost tuning of our alarm had a failure in 24, the one-standard-error rule in 4 and window tuning in 0. Over the thirty random FD001 splits the corresponding counts, of 120 split–ratio cells, were 12 and 4 for optimised policy 1, 20 and 4 with $p_{\text{thres}} = c_p/c_c$, 44 for plain cost tuning, 12 with the one-standard-error rule and 12 for window tuning. In 62% of the five-fold cells with a failure, that failure raised M above 50%. By the decision rule fixed before the run (each plainly optimised policy must fail in more cells than its regularised or heuristic counterpart), the claim that direct cost tuning is fragile was weakened on all four datasets and over the thirty splits. Plain cost tuning of our alarms was clearly more fragile than the one-standard-error rule; for policy 1 the comparison went the other way: with $p_{\text{thres}} = c_p/c_c$ it had a test failure in 19 cells with P-cls and 11 with P-res, against 12 and 4 with optimised thresholds. Window-tuned alarms had a failure in 0 of the 80 five-fold cells but in 12 of the 120 split cells, so the margin that window tuning builds in did not always hold. Measured in cycles, the plainly tuned threshold lay a median of 5.5 cycles above the largest threshold at which a test unit failed, against 9.5 cycles with the one-standard-error rule. The thresholds that minimised the training cost lay below c_p/c_c at the larger ratios (median 0.028 for P-cls on FD001 at $c_p/c_c = 0.1$), as in their simulations, so the heuristic replaced later there. The fragility therefore concerns plain minimisation of an empirical cost over many settings, which a one-standard-error rule mitigated, more than the optimisation of a single probability threshold. The middle and the largest tied thresholds gave different test decisions in 1 of 9,926

cases on FD001–FD004 (largest difference in M 0.2 points) and in none of 8,400 over the thirty FD001 splits, so the tie rule did not drive these results.

Answer to Q3. Optimising the window score produced useful maintenance policies where the alarms were accurate and the test units resembled the training units: window-tuned alarms were cheaper than optimal age replacement in most cost settings on six of seven datasets, but not on N-CMAPSS. It was not equivalent to optimising cost. Tuning the same regressor directly to cost was cheaper on some datasets and more expensive on others; under M , a simple grid-aware probability-threshold policy built on the same regressor had a lower M on FD001 and a higher M on FD002 and FD003; and no tuning rule or policy family was best on every dataset. The window score is therefore an operational specification of when alarms should fire, not a measure of decision value.

8. Transfer to a held-out batch and other asset types (Q4)

The results of Sections 6 and 7 were obtained by cross-validation by unit, in which the test units come from the same population as the training units. This section changes the test population in three ways: a held-out test batch of lithium-ion cells, which changes the test campaign; N-CMAPSS, which changes the operating profiles within the same simulator family; and rolling bearings, a different asset type. On the batteries, where failure is defined by a measured quantity, a simple capacity baseline is evaluated alongside the recipe on accuracy, notice and cost.

8.1. Battery cells: a held-out test batch

Battery cells, within and across batches. Under the common physical task with fully nested preprocessing (Table 11, Fig. 8), the recipe placed 117/123 (95.1%) of first warnings in the window under cross-validation by cell, in which every batch is represented among the training cells (below, “within batches”), 98.4% with κ chosen by inner cross-validation, and 95.1–98.4% for any fixed κ from 2 to 6. With a whole batch held out, the rate fell to 98 of 123 (79.7%; batch 1: 69.4%, batch 2: 88.4%, batch 3: 79.5%). It was 74.8% with κ chosen by inner cross-validation and 63.4–84.6% across fixed κ . Under two conventions that express the task in blocks, the held-out-batch rate was 69.9% (convention A, $\kappa = 3$ from all cells, as in the protocol) and 65.0% (convention B, κ from the outer training cells). The held-out-batch estimate therefore depends on the time-scale convention: the fixed- κ values alone span 21.1 percentage points. Every version lies below every within-batch version and below the protocol criterion of 85%. Paired by cell, the fall from cross-validation to a held-out batch was significant under every convention of the fully nested analysis (exact McNemar; rule: 22 against 3 discordant cells, p

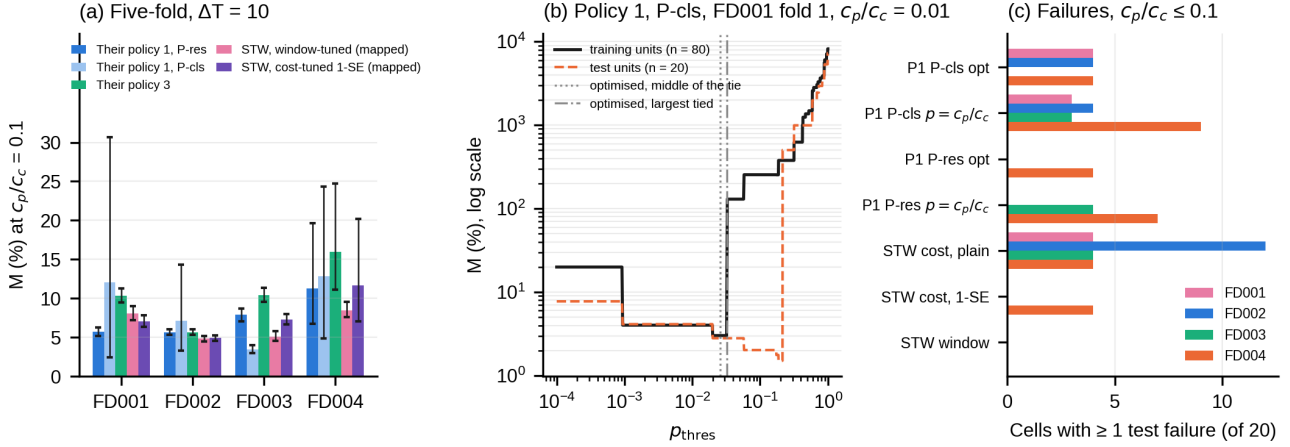


Fig. 7: Comparison with the metric M of Kamariotis et al. ($\Delta T = 10$ cycles, no lead time). (a) M at $c_p/c_c = 0.1$ on FD001–FD004 under five-fold cross-validation, with bootstrap 95% intervals, for their policies 1 and 3 on our regressor and for our alarms mapped to the decision grid. (b) Their Fig. 9a on our data: M of policy 1 (P-cl) as a function of p_{thres} on the training engines (solid) and the test engines (dashed) of FD001 fold 1 at $c_p/c_c = 0.01$; vertical lines: the middle and the largest of the tied optimal thresholds. (c) Number of the 20 fold-ratio cells (five folds, $c_p/c_c \leq 0.1$) with at least one test failure, by policy and dataset.

Table 11

Battery cells under a common physical task ($H = 60$ cycles, window $[-39, +6]$ cycles), fully nested preprocessing and selection (convention C, except where κ is fixed or selected). Correct first warnings, and correct first warnings that also left at least 45 cycles of notice.

Evaluation	Recipe, κ by rule (nested)	Recipe, κ by inner CV	Recipe, κ fixed 2–6 (range, %)	Capacity trend (κ by rule)	Recipe: correct and ≥ 45 cycles left (%)	Trend: correct and ≥ 45 cycles left (%)
Cross-validation by cell	117/123 (95.1)	121/123 (98.4)	95.1–98.4	123/123 (100.0)	80.5	32.5
One batch held out (all)	98/123 (79.7)	92/123 (74.8)	63.4–84.6	121/123 (98.4)	44.7	43.1
Batch 1 held out	25/36 (69.4)	30/36 (83.3)	–	36/36 (100.0)	69.4	77.8
Batch 2 held out	38/43 (88.4)	27/43 (62.8)	–	41/43 (95.3)	11.6	0.0
Batch 3 held out	35/44 (79.5)	35/44 (79.5)	–	44/44 (100.0)	56.8	56.8

123 cells. κ by rule: $\max(1, \text{round}(\text{median training life} / 209.5))$ from each training subset; κ by inner CV: chosen among 2–6 by the summed inner out-of-fold count. Paired exact McNemar, recipe (κ by rule) against the capacity trend: cross-validation 0/6 ($p = 0.031$), held-out batch 2/25 ($p < 0.001$). Task in blocks (cross-validation and held-out batch): convention A ($\kappa = 3$ from all cells, as in the protocol) 97.6% and 69.9%; convention B (κ from the outer training cells) 95.9% and 65.0%, capacity trend 100.0% and 99.2% (Supplementary Sections S5 and S16).

< 0.001; largest p over the κ conventions < 0.001). The batches differ in charging protocols, rest periods and start dates, and in typical life (median 772.5, 481.0 and 1005.5 cycles). Validation within familiar conditions therefore overstated what the recipe achieves on a new test campaign. The within-batch result is an upper reference, and the held-out-batch range is the relevant estimate for a new campaign.

A strong simple baseline. Battery end of life is defined by discharge capacity falling to 0.88 Ah, and capacity is one of the measured inputs. Extrapolating the recent capacity trend to that level, with the same trigger and with settings chosen on the training cells, placed 123/123 (100.0%) of first warnings in the window within batches and 121/123 (98.4%) with a batch held out. That is significantly more than the recipe in both cases (Table 11) and stable across κ (Fig. 8b). Its correct alarms, however, gave less notice. Within batches its median remaining life at the alarm

was 40 cycles against 55 for the recipe, and 32.5% of cells received a correct alarm with at least 45 cycles left, against 80.5%. With a batch held out the two were similar at 45 cycles (43.1% against 44.7%). The trend was ahead at 30 cycles (75.6% against 61.8%) and behind at 60 (4.9% against 29.3%; Fig. 8c). On batch 2, the shortest-lived batch, 88.4% of the recipe's first warnings were correct, but only 11.6% of its cells received a correct alarm with at least 45 cycles left (trend: 95.3% and 0.0%). A higher window score therefore did not mean more usable notice, which is what the cost analysis prices.

Under the cost model (Table 12; Supplementary Section S11 plots the cost against the lead time), with fully nested preprocessing and every threshold and lead time in cycles, the trend was again the cheaper signal. Under cross-validation each of the four trend policies had a lower point estimate than its recipe counterpart in all 16 lead-time and cost-ratio cells; the

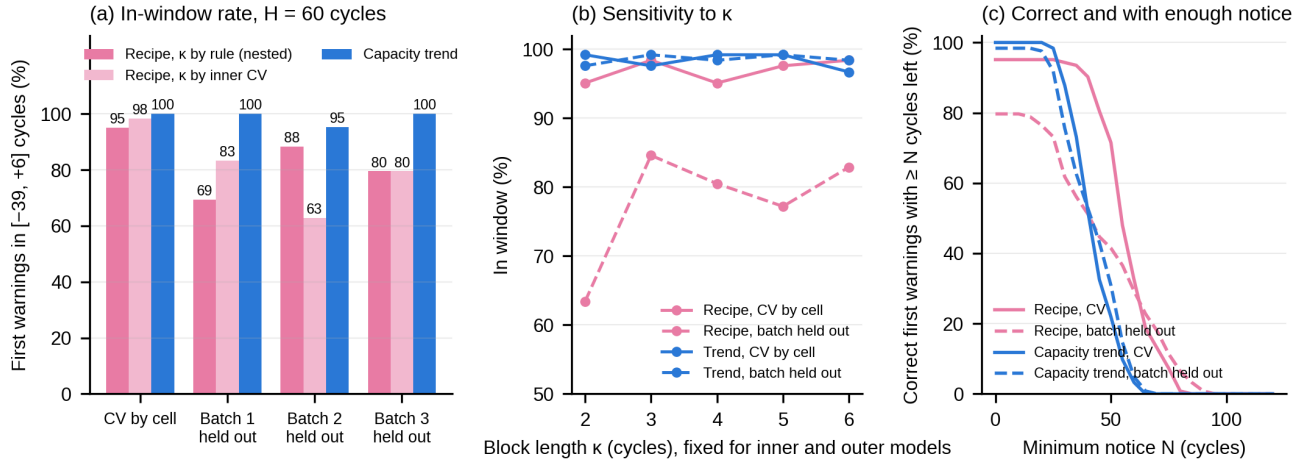


Fig. 8: Battery cells under a common physical task ($H = 60$ cycles, window $[-39, +6]$ cycles) with fully nested preprocessing. (a) In-window rate under cross-validation by cell and with each batch held out: the recipe with κ by the nested rule or by inner cross-validation, and the capacity trend. (b) Sensitivity to a fixed block length κ . (c) Share of cells whose first warning was correct and left at least N cycles before failure.

Table 12

Battery maintenance cost under cross-validation by cell and with one batch held out, fully nested preprocessing with the task in cycles (convention C): long-run cost rate relative to perfect foresight, and in brackets the number of the 123 cells that failed before replacement.

Policy	CV, $\ell = 0$, $\rho = 10$	CV, $\ell = 30$, $\rho = 50$	Batch held out, $\ell = 0$, $\rho = 10$	Batch held out, $\ell = 30$, $\rho = 50$	Batch held out, $\ell = 45$, $\rho = 50$
Recipe, window-tuned	1.14 (1)	1.54 (1)	1.37 (4)	1.10 (0)	1.10 (0)
Recipe, cost-tuned (1-SE)	1.19 (1)	1.09 (0)	1.41 (4)	2.78 (4)	1.51 (1)
Capacity threshold (1-SE)	1.06 (0)	2.01 (2)	1.13 (1)	12.49 (26)	6.16 (11)
Capacity trend, window-tuned	1.02 (0)	1.05 (0)	1.02 (0)	1.05 (0)	1.47 (1)
Capacity trend, cost-tuned (1-SE)	1.05 (0)	1.06 (0)	1.05 (0)	1.48 (1)	1.07 (0)
Age replacement	2.23 (3)	4.57 (2)	6.88 (40)	36.61 (40)	36.61 (40)

ℓ : lead time in charge-discharge cycles; ρ : failure-to-replacement cost ratio. Every inner training subset builds its own blocks (κ by the protocol rule from that subset), z-score statistics and models; warning levels 30–120 cycles, window $[-39, +6]$ cycles, thresholds 3–180 cycles; same splits, inner folds and bootstrap rule as Table 8. Cells of the 16-cell grid in which the cost-tuned capacity trend (1-SE) was significantly cheaper / dearer than the window-tuned recipe: cross-validation 11/0, held-out batch 12/0; window-tuned trend against window-tuned recipe 15/0 and 12/0. Every cell, and the earlier analysis with κ from the outer training cells (convention B), are in Supplementary Section S11.

interval excluded equality in 15 of 16 cells for the window-tuned pair, 11 for the cost-tuned trend with the one-standard-error rule against the window-tuned recipe, 16 against the recipe with the same rule, and 0 for plain cost tuning of both. With a batch held out the corresponding counts were 12, 12, 8 and 16 of 16, and no trend policy was significantly dearer than its recipe counterpart in any cell. The fully nested window-tuned recipe was less fragile on a held-out batch than in the earlier analysis with κ from the outer training cells: it let 4 of 123 cells fail without lead time at $\rho = 10$ and 0 with a lead time of 30 cycles at $\rho = 50$ (earlier 10 and 12), and with a lead time of 45 cycles its point estimate was below that of the window-tuned trend at every cost ratio, without a significant difference. Plain cost tuning of the recipe let 14 cells fail at $\ell = 0$, $\rho = 10$, and 16 at $\ell = 30$, $\rho = 50$, and the capacity threshold with the one-standard-error rule let 26 fail at $\ell = 30$, $\rho = 50$. Age replacement, with its replacement age chosen on the other batches, let 40 cells fail with a batch

held out. On these cells, then, a physically motivated baseline aimed at the failure definition did better than the learned recipe on window accuracy across batches and, in most cost settings, on cost, whereas the recipe gave more notice within batches.

8.2. N-CMAPSS and rolling bearings

N-CMAPSS. The recipe placed 68 of 99 first warnings in the window (68.7%, 95% CI 58.6–77.6%); 18 were late and 5 engines were missed (Table 13). It was clearly better than age-only (24.2%; criterion 3 met), but neither threshold was met, and its advantage over the MLP with Kalman filter (58.6%) was not significant (25 against 15 discordant engines, $p = 0.154$). Lives are short, so a 20-cycle alarm comes at roughly three quarters of life, and the true remaining life at the alarm was spread more widely than on PHM08 (Fig. 5b). N-CMAPSS comes from the same simulator family as the development data, so it tests a change of operating profile rather than of asset type.

Table 13

N-CMAPSS and rolling bearings ($H = 20$; five-fold cross-validation by unit unless stated), with the two competitors fixed in the protocols.

Dataset and analysis	Recipe: correct (%)	95% CI	Late / early / missed	MLP + Kalman filter (%)	Age-only (%)
N-CMAPSS, 99 engines (pre-specified)	68/99 (68.7)	58.6–77.6	18 / 8 / 5	58.6	24.2
N-CMAPSS, official split (39 test engines)	22/39 (56.4)	39.6–72.2	10 / 6 / 1	43.6	30.8
Bearings, 17, protocol ($\kappa = 7$ from all bearings)	0/17 (0.0)	0.0–19.5	0 / 7 / 10	0.0	5.9
Bearings, 17, κ from training bearings	0/17 (0.0)	0.0–19.5	0 / 7 / 10	0.0	5.9
Bearings, official split (11 test), κ from training bearings	1/11 (9.1)	0.2–41.3	0 / 2 / 8	9.1	0.0

Protocol hypotheses at $H = 20$ (in-window $\geq 85\%$; late or missed $\leq 10\%$; better than age-only; better than the MLP with Kalman filter; exact McNemar, two-sided $p < 0.05$): N-CMAPSS, only the third met; bearings, none met. For the battery cells with the task in blocks (conventions A and B), all four were met under cross-validation (Supplementary Section S5).

Bearings. The recipe failed. None of the 17 first warnings fell in the window, either in the protocol analysis or with κ from the training bearings (7 too early and 10 missed). The competitors were also almost always wrong (age-only 1, MLP with Kalman filter 0 of 17). With the physical task fixed ($H = 140$ snapshots), block lengths of 3, 5, 7, 10, 14 snapshots gave at most 1 of 17. The pre-specified fifth hypothesis, that STW-T would be no more than 5 points worse, was met only trivially (0% against 0%), which cannot establish non-inferiority. A post hoc diagnosis with an ad hoc criterion (Supplementary Section S7) found that in 11 of 17 bearings the vibration level rose to and stayed above 1.5 times its early-life level only within the last six blocks before failure (about seven minutes), or never. 6 of these were never warned, which is consistent with failure becoming visible too late for a 20-block alarm. The other 5 received alarms 63–211 blocks before failure, long before any sustained rise, which late visibility cannot explain. Of the 6 bearings with longer degradation, 4 were missed and 2 warned too early. Late onset is therefore at most a partial explanation; small training sets (13–14 bearings per fold), three operating conditions and the feature set are competing explanations that were not tested. The result shows that this recipe, with these features, did not work on these bearings, not that advance warning of bearing failure is impossible.

Answer to Q4. Performance validated within familiar conditions did not transfer. On the battery cells the recipe's in-window rate fell from 95.1% under cross-validation by cell to 79.7% with a batch held out (63.4–84.6% across fixed κ , 74.8% with κ by inner cross-validation, 69.9% and 65.0% under conventions A and B), and a capacity-trend baseline aimed at the failure definition was more accurate and, in most cost settings, cheaper, although it gave less notice within batches. On N-CMAPSS the recipe was weak, and on the bearings it failed. Window accuracy, notice and cost each changed under the change of population, and not by the same amount, so each needs to be re-estimated on held-out batches, campaigns or asset types.

9. Discussion

9.1. What the study establishes

On the same alarms, the five properties of Section 3.2 gave different answers. Long-range accuracy was low and near-failure accuracy high for the same model, so a comparison across observation times measured mainly the difficulty of the prediction problem. The window score of a threshold-crossing alarm was driven by its timing, not by the estimate it reported. Maintenance value depended on the cost model, the decision grid and the dataset, and it was not ranked reliably by the window score. And the accuracy, notice and cost of the alarms changed when the test population changed. The practical consequence is that a single number, whether a window score, a fixed-time error or a cost ratio, cannot certify an end-of-life alarm. The evidence is limited to simulated and laboratory run-to-failure data, one family of threshold-crossing alarms and one acceptance window (Section 9.6), and several results differed between datasets, which is itself part of the finding.

9.2. Why alarm-window scores can mislead

Part of the result is built into the score. For an alarm that fires when a smoothed estimate first crosses a fixed level, the reported estimate at the alarm is close to that level by construction, so the window condition becomes, approximately, a condition on the true remaining life at the alarm. What is not built in is how far this goes and what follows from it. The constant-estimate control measured it: over seven datasets the model's estimate added at most 3.6 percentage points, none significant. Tuning to the window then acts mainly on the alarm time, as the shift study showed. Neither of these makes the score uninformative. On PHM08 and FD001–FD004 it separated a regressor with good timing from linear health indicators with poor timing, even when those were calibrated to cycles and given the same trigger, so it is a meaningful measure of timing at the requested level. The misreading arises when the score is taken as evidence of an accurate

remaining-life trajectory, of adequate notice or of low maintenance cost, none of which it measures.

The observation-time result shows a second way in which a comparison can mislead. A fixed-time model scored at half of life and an alarm scored at its own time are evaluated on prediction problems of different difficulty: on PHM08 the same calibrated model went from 24.8% at half of life to 85.3% at 90% of life, and on FD004 from 18.9% to 66.7%. A large apparent advantage of an alarm over a fixed-time model can therefore come from the time at which accuracy is measured rather than from the alarm. Under matched training and calibration, self-timing added no measurable accuracy over the same regressor at 90% of life on either dataset, and on FD004 it fell short of the regressor at the oracle time of 20 cycles left. Its value is operational: it reaches near-failure accuracy without knowing the life.

9.3. Implications for maintenance decisions

Kamariotis et al. [9] argue that prognostic algorithms and the policies built on them should be judged and tuned by the long-run cost of the decisions they induce. Our results support that view and add three qualifications. First, how the cost is minimised matters. Minimising the empirical cost rate over about two thousand settings sometimes selected alarms with too little margin and produced large losses when failures were expensive; a one-standard-error rule reduced the largest losses, and window tuning was cheaper than that rule in most cells only on FD003. For their policy 1, by contrast, the optimised probability threshold failed on test units less often than the heuristic one, so the fragility concerns plain minimisation over many settings rather than threshold optimisation as such. Second, the decision rule matters. With the same regressor, a threshold on the probability of failure before the next decision time gave a lower M than our self-timed trigger on FD001 and a higher M on FD002 and FD003. Where our trigger lost, it had been chosen for accuracy on the window, not for the notice that the decision grid requires, and it fires between decision times: on FD001, mapping it to the grid raised its M from 3.93% to 8.08%. Where it won, the narrow late side of the window plausibly built in a safety margin, but that margin did not always hold (Section 7.5). Third, the signal can matter more than the tuning criterion: on the battery cells, window-tuned and cost-tuned versions of a capacity trend were cheaper than the corresponding recipe policies in every cost setting under cross-validation and never significantly more expensive with a batch held out.

For reliability engineering practice this means that an alarm score should not be used as a stand-in for decision cost. Where cost ratios, lead times and decision times can be stated, the alarm should be evaluated, and if possible tuned, under them, with a safeguard against

selection noise, and compared with age replacement and with a policy that uses the decision times explicitly. Where they cannot be stated, a window remains a reasonable operational specification of when alarms should fire, because its settings other than the warning level do not depend on uncertain cost ratios, but its score should then be reported as a timing result. When a decision-oriented metric such as M is reported, the decision grid, the perfect reference and the mapping of alarms to decision times should be stated, because they change its value by several percentage points (Section 7.2).

9.4. Transfer and generalisation

The transfer results are the most consequential for practice, because a validation within familiar conditions is what an operator usually has. On the battery cells, the held-out-batch rate of the recipe was 14 to 32 percentage points below the within-batch rate, depending on the time-scale convention, and the held-out estimate itself depended on the convention (63.4–84.6% across fixed κ). The batches differ in charging protocols, rest periods and typical life, which is the kind of difference a new test campaign or production lot brings. A capacity-trend baseline aimed at the failure definition lost little across batches (100.0% to 98.4%), so the loss was specific to the learned recipe, not inherent to the task. On N-CMAPSS, the same simulator family under different operating profiles, the recipe reached 68.7%, and on the bearings, a different asset type, it failed. The bearing result is informative as a negative result: an alarm recipe that works on one asset class gives no assurance for another, and a late onset of visible degradation explained only part of the misses and early alarms (Section 8.2). Within-population cross-validation should therefore be read as an upper reference, and claims of transferability need whole batches, campaigns or asset types held out, with a simple physically motivated baseline where one exists.

9.5. Recommended reporting

From these results we derive a recommendation for what studies of end-of-life alarms should report (Table 14). It is not an established standard and is not complete; it lists the properties that, in this study, could not be inferred from one another, together with the controls that separated them.

9.6. Limitations

- **Scope of the claim.** The evidence concerns one asymmetric window, threshold-crossing alarms built on one gradient-boosted regressor and its variants, and the policies compared here. Alarms of other kinds, such as detectors or policies acting on full predictive distributions, may relate to the window differently, and the five properties may be closer together for other models or data.

Table 14

Recommended reporting for evaluations of end-of-life alarms, derived from this study. Each row is a property that could not be inferred from the others here.

Property	What to report	Why: evidence from this study
A. Long-range prediction	Accuracy at fixed fractions of life or remaining lives, chosen independently of the policy and stated	The same calibrated model placed 24.8% of PHM08 estimates in the window at half of life and 85.3% at 90% of life (Table 5)
B. Near-failure estimation	Accuracy near failure under the same training data and calibration as the alarm, so that alarm and fixed-time results are comparable	The self-timed alarm (91.7%) was no more accurate than its regressor at 90% of life (91.7%) (Table 5)
C. Alarm timing and notice	Distribution of the true remaining life at the alarm; share of correct alarms with the notice the task needs; missed units	A correct alarm at $H = 20$ allowed roughly 9–31 cycles of notice; on the battery cells the policy with the higher window score gave less notice (Sections 6.2, 8.1)
C. Estimate versus timing	A control that keeps the alarm times and replaces the reported estimate by a constant	The constant scored within 3.6 points of the model's estimate on seven datasets (Table 7)
D. Maintenance-policy value	Cost under stated lead times, cost ratios and decision times, against age replacement and a policy tuned directly to cost; for M , the decision grid, the perfect reference and the mapping of alarms to decision times	Rankings changed with the dataset and the probability model; mapping to the grid raised M of the window-tuned alarm on FD001 from 3.93% to 8.08% (Tables 8, 9)
E. Transfer	Results with whole batches, campaigns, operating conditions or asset types held out, against a simple domain baseline	Battery: 95.1% within batches, 79.7% with a batch held out (63.4–84.6% across fixed κ ; 74.8% with κ by inner CV; 69.9% and 65.0% under conventions A and B); the capacity trend did better; bearings 0 of 17 (Tables 11, 13)
All	Selection on training units only, leakage checks, and the status of each analysis (pre-specified, corrected or exploratory), with intervals and what they cover	Several conclusions of this study changed when analyses were corrected or extended (Table 3; Supplementary Section S16)

• **No field data.** Six of the eight datasets are simulations from one simulator family (C-MAPSS), and the battery and bearing data are laboratory measurements; no alarm was deployed. On the batteries, end of life is a threshold on a measured quantity. First-warning rates of the recipe ranged from 0% to 98% across datasets and analyses and fell under a change of test campaign, so they must be re-estimated for each application.

• **Development, pre-specification and corrections.** The recipe and grids were developed on PHM08, and the primary warning level was designated after the C-MAPSS results were known. Only three datasets had protocols fixed in advance; the hashes were not deposited with a registry, so their timing rests on the authors' records, and they do not constitute a formal preregistration. The training-only κ and the battery analyses with a common physical task and fully nested preprocessing are corrections made after the protocol analyses. The matched decompositions, the timing controls, the capacity baselines, the cost analyses, the shift study and the comparison with M are exploratory; for the most recent of them the plan was hashed before the run. The bearing analysis and the battery timing controls and shift study are not fully nested (Section 5.2).

• **Comparison with M .** It is a re-implementation without the reference code, splits or prognostic models; their published version should be checked against the preprint used here for any change to the policy definitions. Their policies need a failure probability,

which we built from our regressor in two ways, and the verdicts differed between the two. Our alarms fire between decision times and had to be mapped to a grid of 10 cycles, which their policies use by construction; the grid also changes the perfect reference, by about 2% on the C-MAPSS datasets and by more than 7% on N-CMAPSS, which we therefore excluded. Only the replacement-only setting without lead time was examined, the cost ratios were chosen to span the ranges shown in their figures, and their caution about evaluations on 20 test units applies to our thirty random splits and to any comparison with their reported values. FD002–FD004 were evaluated by five-fold cross-validation only.

• **Scope of the decomposition.** The matched comparison of Q1 uses two datasets of one simulator family (PHM08 and FD004), one fixed-time model and one recipe. Differences of a few points could go undetected with about 220–250 engines.

• **Competitors and baselines.** The challenge approaches are re-implementations, not the original code, and only the networks' caps and warning settings were tuned. The health-indicator, calibrated-indicator, capacity-threshold and capacity-trend baselines are single representatives of their classes, and the capacity trend exists only for the batteries.

• **Statistics.** Intervals are conditional on the fitted policies; refitting and fold-assignment variability is reflected only in the repeated PHM08 and FD001 splits and the κ sensitivity. Non-significant differences are not evidence of equivalence. Multiplicity

is adjusted within tables, not across the paper, and the cost cells or cost ratios within a dataset are correlated, so they are not replications. The interval of their Eq. 7 ignores the variability of the perfect reference and was not used for our verdicts.

- **Cost model.** The renewal–reward model assumes a fixed lead time, independent units and replacement as good as new. The cost ratios are illustrative, and the model ignores spare parts, downtime and the grouping of maintenance across units [8, 23]. The window-tuned and cost-tuned policies select on the same out-of-fold predictions that tune their other settings.
- **Theory.** Proposition 1 holds for a fixed error distribution. For the sequential policy it is only an approximate description on these data, with errors of up to 8.0 points within ± 4 of the selected shift, not a guarantee.

9.7. Open questions

Five questions remain open. **Decision-aware alarms:** a trigger could choose its warning level, or its threshold on a failure probability, for the decision times and lead time of the maintenance task, and be compared with the policies of Kamariotis et al. [9] in their ordering setting with lead time. **Field data and changing conditions:** whether recalibrating the shift on the first failures of a new population, or attaching distribution-free uncertainty to the alarm time, preserves accuracy and cost needs fleet data with censoring. **Maintenance policies:** the alarms of several units could be grouped into shared visits [8, 23], with the comparisons of window, cost and simple baselines repeated under such policies. **Test and inspection policies:** for units that are proof-tested or inspected at intervals [2], an alarm could trigger an additional test rather than a replacement, which requires a cost model with test costs. **Fault detection:** for failures that become visible only shortly before they happen, as on several bearings, the first-warning rule should be compared with sequential detectors such as cumulative-sum and exponentially weighted moving-average charts [33, 34] under the same leakage-safe protocol.

10. Conclusion

This study asked what an end-of-life alarm score measures and whether optimising it yields good maintenance decisions. On eight run-to-failure datasets, with every setting selected on training units only, long-range prediction, near-failure estimation, alarm timing, maintenance-policy value and transfer behaved as distinct properties. Observation time explained most of the contrast between fixed-time and alarm evaluation (on PHM08, 24.8% at half of life against 85.3% at

90% of life for the same calibrated model), and self-timing added no measurable accuracy. The asymmetric window score was driven by alarm timing: a constant estimate at the same alarm times scored within 3.6 percentage points of the model's estimate, and the score revealed neither the notice given nor the cost incurred.

Optimising the window was not equivalent to optimising cost. Window-tuned alarms were cheaper than age replacement in most cost settings on six of seven datasets, but direct cost tuning was better on some datasets and worse on others, and under the re-implemented metric of Kamariotis et al. their probability-threshold policy on our regressor had a lower M than our window-tuned alarm on FD001 (5.69% against 8.08% on FD001, the dataset of their study) and a higher M on FD002 and FD003; no policy was best everywhere. Performance validated within familiar conditions did not transfer: on battery cells with fully nested preprocessing the recipe's in-window rate fell from 95.1% to 79.7% with a batch held out (63.4–84.6% across fixed time scales; 74.8%, 69.9% and 65.0% under the other conventions), a capacity-trend baseline did better, and the recipe failed on bearings (0 of 17).

An end-of-life alarm should therefore not be judged by a single score. Studies should report its accuracy far from and near failure, the notice its alarms give, a control that separates the estimate from the timing, its cost under stated decision conditions against age replacement and a cost-tuned policy, and its performance on held-out batches, campaigns or asset types against a simple domain baseline. What remains unresolved is how these properties behave on field data with censoring and changing conditions, and whether alarms designed for the decision times and lead times of the maintenance task close the gap between timing and cost. These conclusions rest on simulations and laboratory tests and support a way of evaluating alarms, not the deployment of the recipe.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used Claude (Anthropic) in order to assist with code development, data analysis and drafting of the manuscript. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

Data availability

The PHM08, C-MAPSS and N-CMAPSS datasets are available from the NASA Prognostics Center of Excellence data repository, the battery data of Severson et al.

[19] from <https://data.matr.io>, and the bearing data of the IEEE PHM 2012 challenge [18] from the FEMTO-ST institute. Code, hashed protocols, derived data summaries and the result files needed to reproduce every number are in a repository whose link is withheld for anonymised review.

References

- [1] Barlow R, Hunter L. Optimum preventive maintenance policies. *Operations Research* 1960;8(1):90–100. <https://doi.org/10.1287/opre.8.1.90>
- [2] de Jonge B, Scarf PA. A review on maintenance optimization. *European Journal of Operational Research* 2020;285(3):805–824. <https://doi.org/10.1016/j.ejor.2019.09.047>
- [3] Lei Y, Li N, Guo L, Li N, Yan T, Lin J. Machinery health prognostics: A systematic review from data acquisition to RUL prediction. *Mechanical Systems and Signal Processing* 2018;104:799–834. <https://doi.org/10.1016/j.ymssp.2017.11.016>
- [4] Si X-S, Wang W, Hu C-H, Zhou D-H. Remaining useful life estimation – A review on the statistical data driven approaches. *European Journal of Operational Research* 2011;213(1):1–14. <https://doi.org/10.1016/j.ejor.2010.11.018>
- [5] Ramasso E, Saxena A. Performance benchmarking and analysis of prognostic methods for CMAPSS datasets. *International Journal of Prognostics and Health Management* 2014;5(2). <https://doi.org/10.36001/ijphm.2014.v5i2.2236>
- [6] Saxena A, Goebel K, Simon D, Eklund N. Damage propagation modeling for aircraft engine run-to-failure simulation. In: 2008 International Conference on Prognostics and Health Management. IEEE; 2008, p. 1–9. <https://doi.org/10.1109/PHM.2008.4711414>
- [7] Rengasamy D, Rothwell B, Figueredo GP. Asymmetric loss functions for deep learning early predictions of remaining useful life in aerospace gas turbine engines. In: 2020 International Joint Conference on Neural Networks (IJCNN). IEEE; 2020, p. 1–7. <https://doi.org/10.1109/IJCNN48605.2020.9207051>
- [8] de Pater I, Reijns A, Mitici M. Alarm-based predictive maintenance scheduling for aircraft engines with imperfect remaining useful life prognostics. *Reliability Engineering & System Safety* 2022;221:108341. <https://doi.org/10.1016/j.res.2022.108341>
- [9] Kamariotis A, Tatsis K, Chatzi E, Goebel K, Straub D. A metric for assessing and optimizing data-driven prognostic algorithms for predictive maintenance. *Reliability Engineering & System Safety* 2024;242:109723. <https://doi.org/10.1016/j.res.2023.109723>
- [10] Wang T, Yu J, Siegel D, Lee J. A similarity-based prognostics approach for remaining useful life estimation of engineered systems. In: 2008 International Conference on Prognostics and Health Management. IEEE; 2008, p. 1–6. <https://doi.org/10.1109/PHM.2008.4711421>
- [11] Heimes FO. Recurrent neural networks for remaining useful life estimation. In: 2008 International Conference on Prognostics and Health Management. IEEE; 2008, p. 1–6. <https://doi.org/10.1109/PHM.2008.4711422>
- [12] Peel L. Data driven prognostics using a Kalman filter ensemble of neural network models. In: 2008 International Conference on Prognostics and Health Management. IEEE; 2008, p. 1–6. <https://doi.org/10.1109/PHM.2008.4711423>
- [13] Zheng S, Ristovski K, Farahat A, Gupta C. Long short-term memory network for remaining useful life estimation. In: 2017 IEEE International Conference on Prognostics and Health Management (ICPHM). IEEE; 2017, p. 88–95. <https://doi.org/10.1109/ICPHM.2017.7998311>
- [14] Li X, Ding Q, Sun J-Q. Remaining useful life estimation in prognostics using deep convolution neural networks. *Reliability Engineering & System Safety* 2018;172:1–11. <https://doi.org/10.1016/j.res.2017.11.021>
- [15] Arias Chao M, Kulkarni C, Goebel K, Fink O. Aircraft engine run-to-failure dataset under real flight conditions for prognostics and diagnostics. *Data* 2021;6(1):5. <https://doi.org/10.3390/data6010005>
- [16] Grinsztajn L, Oyallon E, Varoquaux G. Why do tree-based models still outperform deep learning on typical tabular data? *Advances in Neural Information Processing Systems* 2022;35:507–520. <https://doi.org/10.52202/068431-0037>
- [17] Friedman JH. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics* 2001;29(5):1189–1232. <https://doi.org/10.1214/aos/1013203451>
- [18] Nectoux P, Gouriveau R, Medjaher K, Ramasso E, Chebel-Morello B, Zerhouni N, Varnier C. PRONOSTIA: An experimental platform for bearings accelerated degradation tests. In: IEEE International Conference on Prognostics and Health Management (PHM'12), Denver, CO, United States; 2012. <https://hal.science/hal-00719503>
- [19] Severson KA, Attia PM, Jin N, Perkins N, Jiang B, Yang Z, Chen MH, Aykol M, Herring PK, Fraggadakis D, Bazant MZ, Harris SJ, Chueh WC, Braatz RD. Data-driven prediction of battery cycle life before capacity degradation. *Nature Energy* 2019;4(5):383–391. <https://doi.org/10.1038/s41560-019-0356-8>
- [20] Saxena A, Celaya J, Saha B, Saha S, Goebel K. Metrics for offline evaluation of prognostic performance. *International Journal of Prognostics and Health Management* 2010;1(1). <https://doi.org/10.36001/ijphm.2010.v1i1.1336>
- [21] Gebraeel NZ, Lawley MA, Li R, Ryan JK. Residual-life distributions from component degradation signals: A Bayesian approach. *IIE Transactions* 2005;37(6):543–557. <https://doi.org/10.1080/07408170590929018>
- [22] Nguyen KTP, Medjaher K. A new dynamic predictive maintenance framework using deep learning for failure prognostics. *Reliability Engineering & System Safety* 2019;188:251–262. <https://doi.org/10.1016/j.res.2019.03.018>
- [23] Mitici M, de Pater I, Barros A, Zeng Z. Dynamic predictive maintenance for multiple components using data-driven probabilistic RUL prognostics: The case of turbofan engines. *Reliability Engineering & System Safety* 2023;234:109199. <https://doi.org/10.1016/j.res.2023.109199>
- [24] Lee J, Mitici M. Deep reinforcement learning for predictive aircraft maintenance using probabilistic remaining-useful-life prognostics. *Reliability Engineering & System Safety* 2023;230:108908. <https://doi.org/10.1016/j.res.2022.108908>
- [25] Xie Z, Abdin A, Fang Y. Toward decision-oriented prognostics: An integrated estimate-optimize framework for predictive maintenance. *arXiv preprint* 2025;arXiv:2506.19698. <https://doi.org/10.48550/arXiv.2506.19698>
- [26] Varma S, Simon R. Bias in error estimation when using cross-validation for model selection. *BMC Bioinformatics* 2006;7:91. <https://doi.org/10.1186/1471-2105-7-91>
- [27] Kapoor S, Narayanan A. Leakage and the reproducibility crisis in machine-learning-based science. *Patterns* 2023;4(9):100804. <https://doi.org/10.1016/j.patter.2023.100804>
- [28] Nosek BA, Ebersole CR, DeHaven AC, Mellor DT. The preregistration revolution. *Proceedings of the National Academy of Sciences* 2018;115(11):2600–2606. <https://doi.org/10.1073/pnas.1708274114>

- 1777 [29] Pedregosa F, Varoquaux G, Gramfort A, Michel V, Thirion B,
1778 Grisel O, Blondel M, Prettenhofer P, Weiss R, Dubourg V,
1779 VanderPlas J, Passos A, Cournapeau D, Brucher M, Perrot M,
1780 Duchesnay E. Scikit-learn: Machine learning in Python.
1781 Journal of Machine Learning Research 2011;12:2825–2830.
- 1782 [30] Clopper CJ, Pearson ES. The use of confidence or fiducial
1783 limits illustrated in the case of the binomial. Biometrika
1784 1934;26(4):404–413.
1785 <https://doi.org/10.1093/biomet/26.4.404>
- 1786 [31] McNemar Q. Note on the sampling error of the difference
1787 between correlated proportions or percentages. Psychometrika
1788 1947;12(2):153–157. <https://doi.org/10.1007/BF02295996>
- 1789 [32] Holm S. A simple sequentially rejective multiple test
1790 procedure. Scandinavian Journal of Statistics
1791 1979;6(2):65–70. <https://www.jstor.org/stable/4615733>
- 1792 [33] Page ES. Continuous inspection schemes. Biometrika
1793 1954;41(1–2):100–115.
1794 <https://doi.org/10.1093/biomet/41.1-2.100>
- 1795 [34] Roberts SW. Control chart tests based on geometric moving
1796 averages. Technometrics 1959;1(3):239–250.
1797 <https://doi.org/10.1080/00401706.1959.10489860>